QCon
全球软件开发大会

InfoQ 技术大会

Code Agent 的 Loop 工程实践

网易智企 CodeWave 的探索与落地

赵雨森

网易智企 · 技术专家

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 企业级应用 L4 全自主生成的背景与挑战

02 需求工程和 Spec Driven 的主要思路

03 如何让 Agent 大规模长程任务高质量运行?

04 如何加速 Code Agent 效果的迭代升级?

05 总结与展望

01

企业级应用 L4 全自主生成的背景与挑战

AI Coding 自主度与成熟度的趋势

> 《The Five Levels: from Spicy Autocomplete to the Dark Factory》—— Dan Shapiro

AI 越来越自主，编程工作由“人主导”逐步走向“AI 主导”。

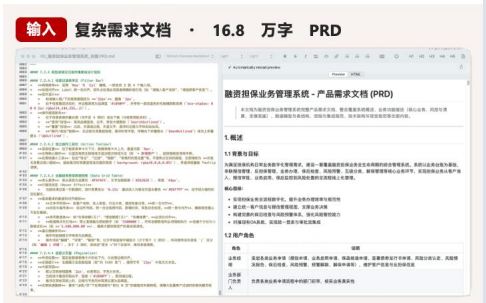
Level	类比	主要特征	描述	主要年份	工具
L0	手动	人工编码	完全由人写代码	2022以前	
L1	巡航控制和车道保持	代码补全、生成	人写代码，AI辅助	2023	GitHub Copilot、Tabnine 等
L2	高速公路自动驾驶	结对编程	AI对话生成，人工检查	2024	Cursor 等以 Code 为主的 IDE
L3	有安全员的自动驾驶	半自主Agent、人机协同	AI主导，人工监督关键点和结果 成熟的 Vibe Coding	2025	Claude Code、Codex 等 CLI/IDE 形态的 Agent
L4	无人驾驶出租车	全自主Agent、需求驱动	AI端到端完成，人类验收结果，无需干预	2026	CodeWave、Replit、Augment Code
L5	黑灯工厂	AI开发团队	多Agent协作、完成团队级任务		暂不讨论

AI 时代应用需求的分化、L3-L4 的对比

Level	轻型应用	重型企业级应用
	• 个人或团队内的应用 • 创意性高的应用、或Demo演示 • 需求宽松、部署要求低	• 生产级、复杂度高、部署链路长、需求严格的应用 • 参考百万企业客户的真实场景
L3 工具 **（Vibe Coding 为主）**	准L4效果	L3效果
L4 工具 **CodeWave 等**	L4效果	L4效果

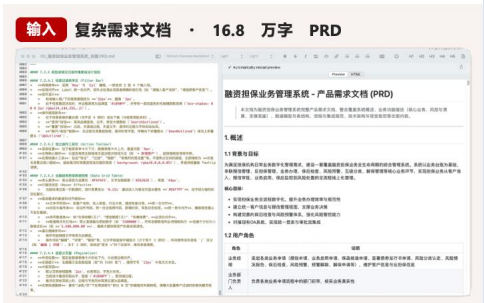
AI 时代应用需求的分化、L3-L4 的对比

Level	轻型应用	重型企业级应用
	• 个人或团队内的应用 • 创意性高的应用、或Demo演示 • 需求宽松、部署要求低	• 生产级、复杂度高、部署链路长、需求严格的应用 • 参考百万企业客户的真实场景 可承接主体 —— 有 IT 开发团队、需要「交付出一个应用」的企业 **50 万+ 家** **规上工业企业** 国家统计局（2025）· 年主营收入 2000 万元及以上 **75 万+ 家** **软件行业活跃企业（含 ISV）** 中国软件行业协会（2025）
L3 工具 **（Vibe Coding 为主）**	准L4效果	L3效果
L4 工具 **CodeWave 等**	L4效果	L4效果



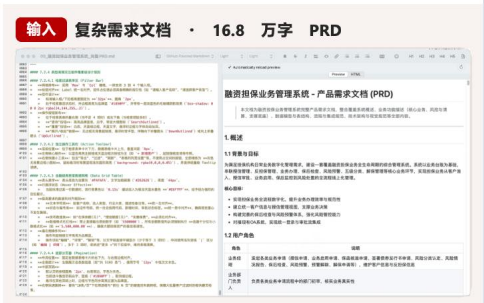
AI 时代应用需求的分化、L3-L4 的对比

Level	轻型应用	重型企业级应用
	• 个人或团队内的应用 • 创意性高的应用、或Demo演示 • 需求宽松、部署要求低	• 生产级、复杂度高、部署链路长、需求严格的应用 • 参考百万企业客户的真实场景 可承接主体 —— 有 IT 开发团队、需要「交付出一个应用」的企业 **50 万+ 家** **规上工业企业** 国家统计局（2025）· 年主营收入 2000 万元及以上 **75 万+ 家** **软件行业活跃企业（含 ISV）** 中国软件行业协会（2025）
L3 工具 **（Vibe Coding 为主）**	准L4效果	L3效果 • 人力无法进一步释放：人工对话多轮才能达到预期、交付链路需要大量人工衔接 • 效果和质量对个体开发者的能力依赖较高、非程序岗赋能不足 • AI 能力个体散点化、缺乏可复用的管线沉淀。同类需求却又重复造轮子
L4 工具 **CodeWave 等**	L4效果	L4效果



AI 时代应用需求的分化、L3-L4 的对比

Level	轻型应用	重型企业级应用
	• 个人或团队内的应用 • 创意性高的应用、或Demo演示 • 需求宽松、部署要求低	• 生产级、复杂度高、部署链路长、需求严格的应用 • 参考百万企业客户的真实场景 可承接主体 —— 有 IT 开发团队、需要「交付出一个应用」的企业 **50 万+ 家** **规上工业企业** 国家统计局（2025）· 年主营收入 2000 万元及以上 **75 万+ 家** **软件行业活跃企业（含 ISV）** 中国软件行业协会（2025）
L3 工具 **（Vibe Coding 为主）**	准L4效果	L3效果
L4 工具 **CodeWave 等**	L4效果	L4效果



- 人力无法进一步释放：人工对话多轮才能达到预期、交付链路需要大量人工衔接
- 效果和质量对个体开发者的能力依赖较高、非程序岗赋能不足
- AI 能力个体散点化、缺乏可复用的管线沉淀。同类需求却又重复造轮子

L4 全自主生成-面临的主要挑战和解决思路

需求工程问题

大规模需求、需求标准化以及后续遵循

如何进一步释放人力?

让大规模 Agent 长程任务高质量运行?

如何进一步提升生成效果?

加速 Code Agent 的迭代升级?

L4 全自主生成-面临的主要挑战和解决思路

需求工程问题

大规模需求、需求标准化以及后续遵循

引入 Spec Driven

如何进一步释放人力?

让大规模 Agent 长程任务高质量运行?

如何进一步提升生成效果?

加速 Code Agent 的迭代升级?

L4 全自主生成-面临的主要挑战和解决思路

需求工程问题

大规模需求、需求标准化以及后续遵循

引入 **Spec Driven**

如何进一步释放人力?

让大规模 Agent 长程任务高质量运行?

通过**Harness + Loop 工程**,
优化 Agent 内部的执行流程

如何进一步提升生成效果?

加速 Code Agent 的迭代升级?

L4 全自主生成-面临的主要挑战和解决思路

需求工程问题

大规模需求、需求标准化以及后续遵循

引入 **Spec Driven**

如何进一步释放人力?

让大规模 Agent 长程任务高质量运行?

通过**Harness + Loop 工程**,
优化 Agent 内部的执行流程

如何进一步提升生成效果?

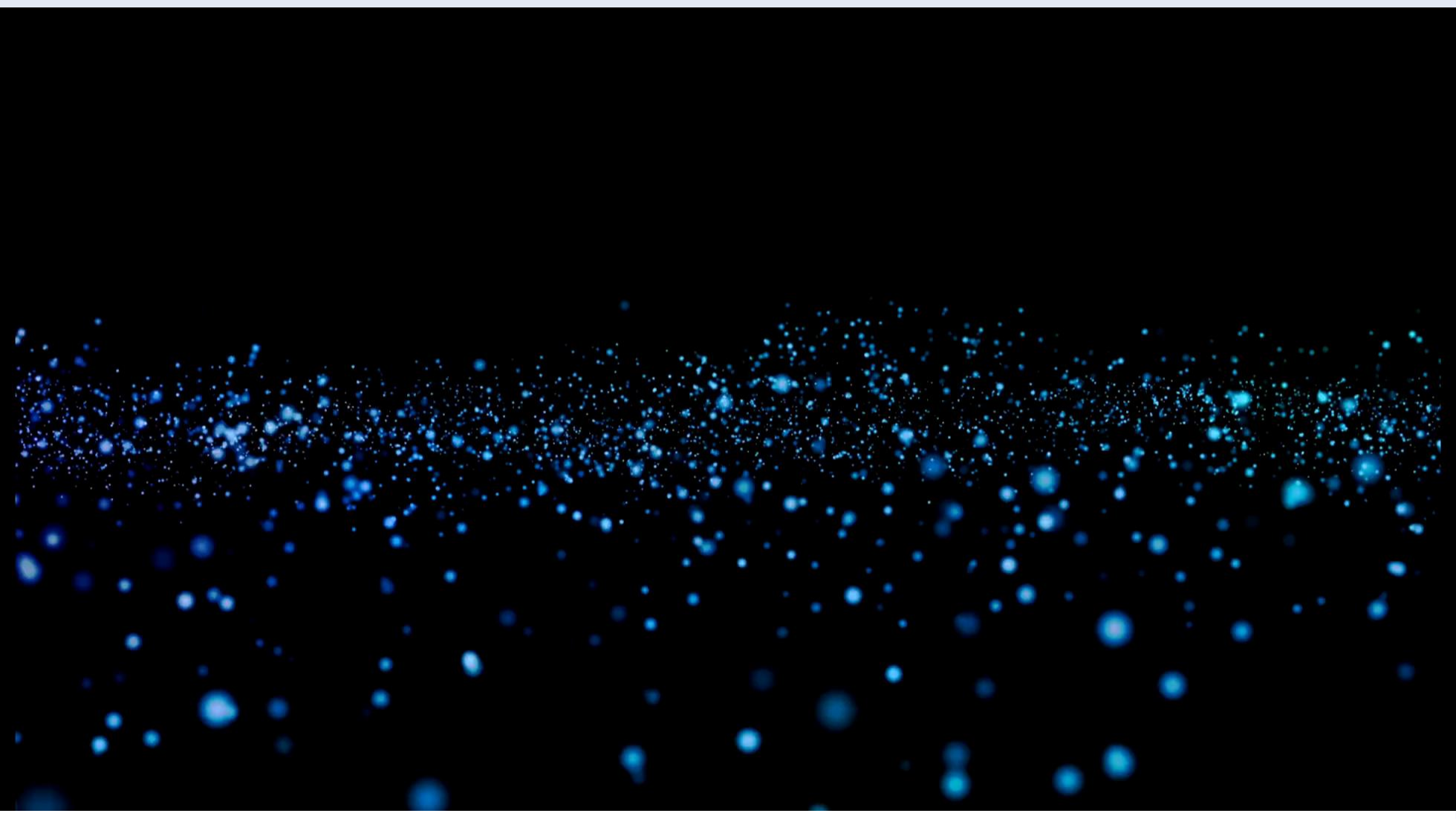
加速 Code Agent 的迭代升级?

通过**Loop 工程**,
优化 Agent 产品研发的迭代流程

02

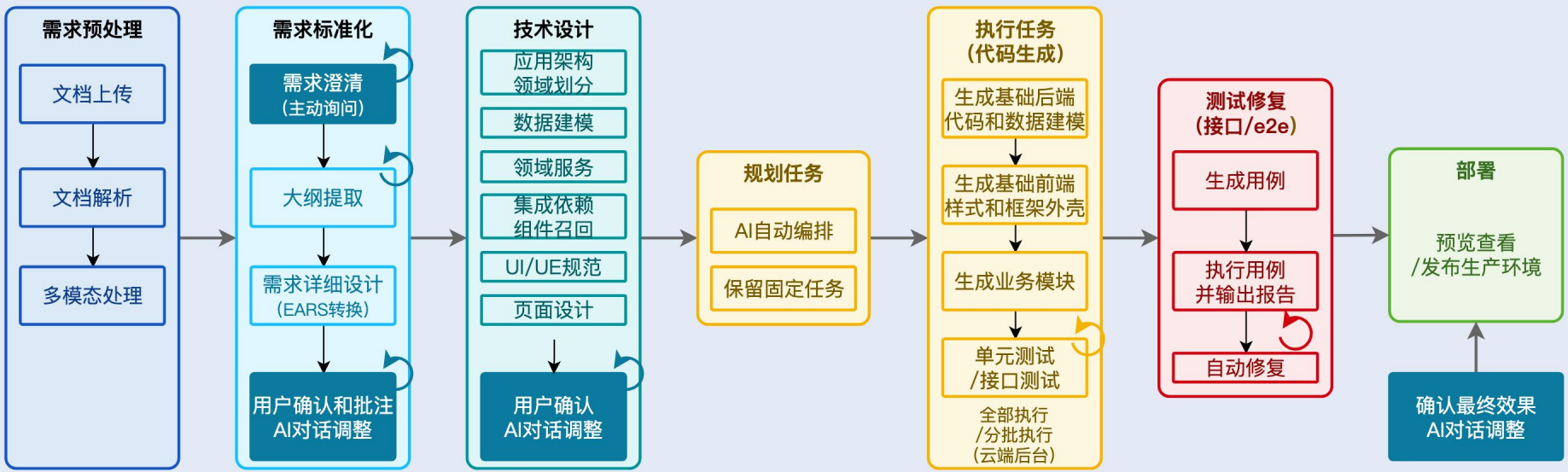
产品思路

Spec Driven 和需求工程



基于 Spec-Driven 的企业级 L4 开发平台

- **全链路 AI 自主**：需求预处理 → 需求标准化 → 技术设计 → 规划任务 → 代码生成 → 自动化验证与修复 → 部署
- **规格驱动**（CodeWave 的理念内核）：先冻结规格、再生成代码，规格与产物逐条可对账



SDD 需求工程的首要环节：如何标准化需求描述？

用户上传的需求文档五花八门，而且我们日常的 PRD 也存在很多歧义和模糊性。

需求阶段不只是文档转md或者换一种写法。如果用户在需求环节没有有效消除歧义（有效减熵），那么后面环节全部会出现偏移

如何标准化需求描述?

SDD 需求工程的首要环节：如何标准化需求描述？

用户上传的需求文档五花八门，而且我们日常的 PRD 也存在很多歧义和模糊性。

需求阶段不只是文档转md或者换一种写法。如果用户在需求环节没有有效消除歧义（有效减熵），那么后面环节全部会出现偏移

如何标准化需求描述？

https://makerneo.com/zh/articles/what-is-ears-requirements-syntax-how-to-write-better-ai-prompts.html

> **EARS（Easy Approach to Requirements Syntax，需求语法简易方法）** 是一种**轻量级、结构化**的方法，旨在通过规范化的自然语言表达来改进需求编写。它由 Alistair Mavin 和 Rolls-Royce PLC 团队于 2009 年开发，最初用于分析航空发动机控制系统的适航性法规。

在 <可选的前提条件> 的情况下，当 <可选的触发条件> 时，<系统名称> 必须 <系统响应>。

示例：

传统描述：用户编辑内容后，系统最好能自动保存。

EARS改进：当 用户在编辑框中停止输入超过5秒时，系统应 自动保存当前内容至草稿箱。

SDD 需求工程的首要环节：如何标准化需求描述？

用户上传的需求文档五花八门，而且我们日常的 PRD 也存在很多歧义和模糊性。

需求阶段不只是文档转md或者换一种写法。如果用户在需求环节没有有效消除歧义（有效减熵），那么后面环节全部会出现偏移

如何标准化需求描述?

https://makerneo.com/zh/articles/what-is-ears-requirements-syntax-how-to-write-better-ai-prompts.html

> **EARS（Easy Approach to Requirements Syntax，需求语法简易方法）** 是一种**轻量级、结构化**的方法，旨在通过规范化的自然语言表达来改进需求编写。它由 Alistair Mavin 和 Rolls-Royce PLC 团队于 2009 年开发，最初用于分析航空发动机控制系统的适航性法规。

在 <可选的前提条件> 的情况下，当 <可选的触发条件> 时，<系统名称> 必须 <系统响应>。

示例：

传统描述：用户编辑内容后，系统最好能自动保存。

EARS改进：当 用户在编辑框中停止输入超过5秒时，系统应 自动保存当前内容至草稿箱。

	传统描述	EARS
消除模糊词	“最好能”“方便一点”	“应”“必须”
省略触发时机	省略触发时机	“当...时”“若...”
拆分复合需求	一句话混多个要求	分多条描述（验证+提示）
量化非功能需求	“不能太慢”	“2秒内”

主要优势

1. 人机沟通方式标准化：不管刚开始是谁写的 PRD，AI 理解和生成稳定
2. 更容易发现原始需求的一些问题，帮助用户进一步澄清
3. 通过结构化 checklist 的手段，可以进行量化评估和验收对账
4. 统一后续的输入，容易设计 benchmark 评估上下游 AI 效果

SDD 需求工程的首要环节：如何标准化需求描述？

安全管理-安全风险双控管理-隐患点检-隐患点检计划.md
安全管理-安全风险双控管理-隐患点检-隐患点检清单.md
安全管理-安全风险双控管理-隐患点检-隐患点检任务表.md
安全管理-安全风险双控管理-隐患点检-隐患点检项记录.md
安全管理-安全风险双控管理-隐患点检-隐患点检项维护.md
安全管理-安全风险双控管理-隐患排查-评安全分.md
安全管理-安全风险双控管理-隐患排查-隐患上报.md
安全管理-安全风险双控管理-隐患排查-隐患上报台账.md
安全管理-安全风险双控管理-专项检查-检查表维护.md
安全管理-安全风险双控管理-专项检查-专项检查计划.md
安全管理-安全风险双控管理-专项检查-专项检查清单.md
安全管理-安全风险双控管理-专项检查-专项检查任务表.md
安全管理-车间管理-车间基础信息管理.md
安全管理-车间管理-车间平面图绘制.md
安全管理-车间管理-车间人员管理.md
安全管理-车间管理-车间危险源管理.md
安全管理-车间管理-风险评价维护.md
劳保用品管理-劳保用品清单.md
通知公告.md
系统管理-变更管理-部门信息变更.md
系统管理-变更管理-人员信息变更.md
系统管理-组织架构-部门信息.md
系统管理-组织架构-厂区.md
系统管理-组织架构-车间信息.md
系统管理-组织架构-风险清单管理.md
系统管理-组织架构-角色管理.md
系统管理-组织架构-人员维护.md

```
## 验收列表

- **隐患上报列表页数据展示与查询**：
  - **数据展示**：
    - 在**当前页面**，展示隐患上报记录列表表格，表头依次包含：隐患编号、隐患描述、隐患分级、隐患位置、隐患车间、隐患类型、发现时间、车间负责人、车间安全员、上报人、状态、操作
    - 在**当前页面**，表格下方左侧显示数据总数统计文案「共 N 条数据」，右侧显示分页控件，包含上一页、页码列表、下一页按钮
  - **查询筛选**：
    - 在**当前页面**，查询条件区依次包含：「隐患类型」下拉框（选项：电气、消防、化学品、设备、人的不安全行为、职业健康、5S、管理不当、其他）、「隐患分级」下拉框（选项：高风险、一般）、「隐患位置」输入框（占位符"请输入隐患位置"）、「隐患状态」下拉框（选项：待上报、待受理、待整改、已整改、已完成）
    - 在**当前页面**，点击「查询」按钮后，系统根据输入的查询条件重新加载表格数据并刷新分页信息
  - **分页操作**：
    - 在**当前页面**，点击分页控件的页码或上一页/下一页按钮后，系统加载对应页码的表格数据并更新当前选中页码的高亮状态

- **隐患上报列表页用户操作**：
  - **发起上报**：
    - 在**当前页面**，点击「隐患上报」按钮后，**跳转到「隐患上报流程页」**进入隐患上报表单填写
  - **撤回操作**：
    - 在**当前页面**，当隐患记录的**状态**为"待受理"时，操作列显示「撤回」按钮；点击「撤回」按钮后，系统将该隐患记录的**状态**回退为"待上报"，并刷新当前表格数据
    - 在**当前页面**，当隐患记录的**状态**不为"待受理"时，操作列不显示「撤回」按钮

- **隐患上报表单填写与提交**：
  - **表单字段展示**：
    - 在**当前页面**的「I 隐患上报」模块中，展示以下字段：「上报人」（纯文本展示当前登录用户姓名，不可编辑）、「发现时间」（带红色*必填标识，日期选择框，默认填充当前日期，可修改）、「是否本车间隐患」（带红色*必填标识，单选框组"是/否"，默认选中"是"）、「隐患车间」（带红色*必填标识，下拉选择框）、「隐患类型」（带红色*必填标识，下拉选择框，选项：电气、消防、化学品、设备、人的不安全行为、职业健康、5S、管理不当、其他）、「隐患厂区」（带红色*必填标识，下拉选择框，来源厂区数据，只能选一个厂区）、「隐患位置」（带红色*必填标识，文本输入框）、「隐患照片」（图片上传组件，非必填）、「隐患描述」（带红色*必填标识，多行文本输入框）
```

SDD 需求工程：解析各种原始需求文档有哪些坑？

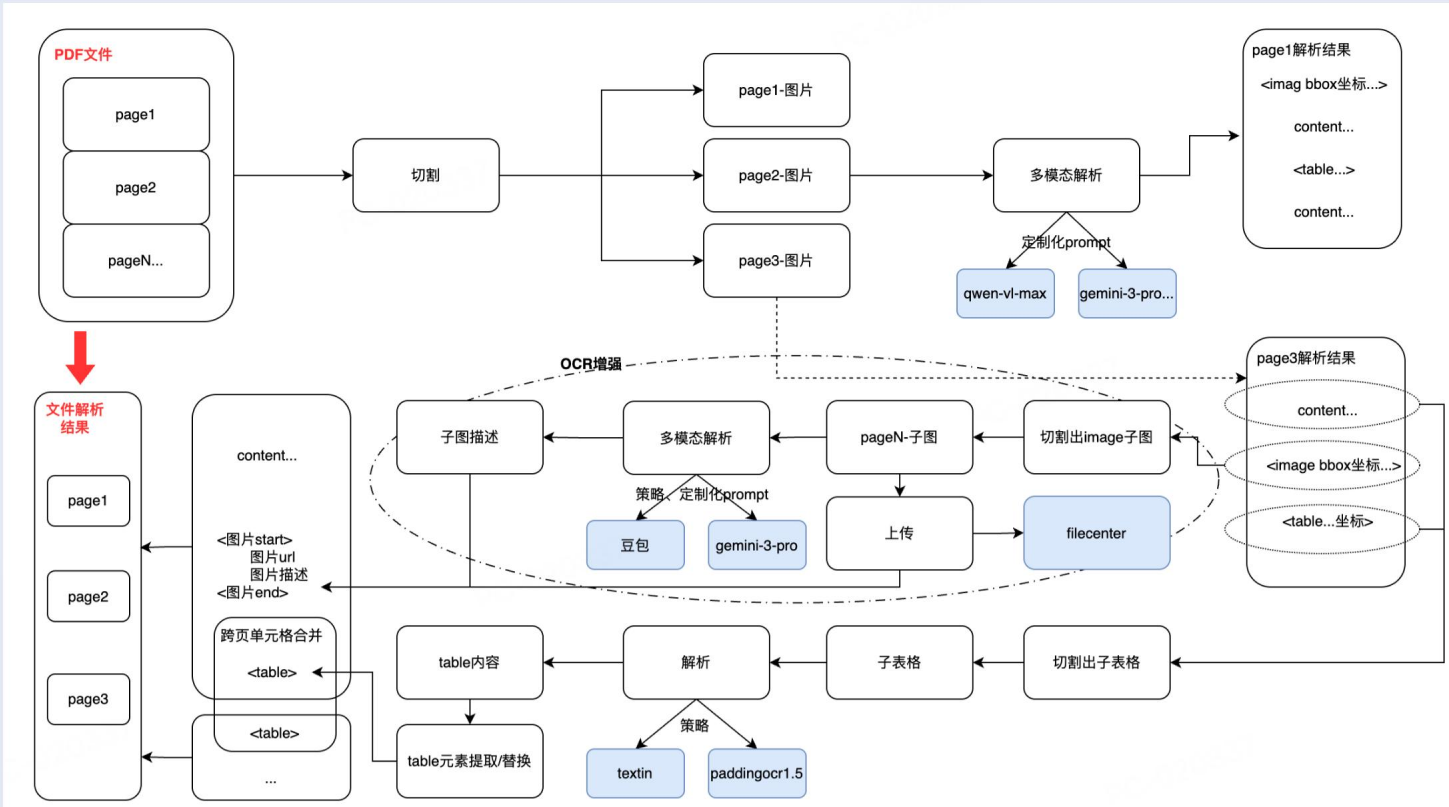
一些商业化解析产品存在问题：

1. 图片识别极不准确：流程图常被误判为表格、图片变乱码等
2. 复杂表格处理难：无法还原有合并单元格的复杂报表、跨页丢失错误等
3. 版面泛化弱：多栏、异形排版阅读顺序易错乱，正文标题误判
4. 无法定制图片解析方式，如流程图应转为mermaid，UI原型图应转为视觉交互描述等

解决方案

- 多种模型和策略组合切割解析
- 多页并行提速
- 兼顾性能、成本、准确率

效果：73.6% → 95%



SDD 需求工程：解析各种原始需求文档有哪些坑？

一些商业化解析产品存在问题：

1. 图片识别极不准确：流程图常被误判为表格、图片变乱码等
2. 复杂表格处理难：无法还原有合并单元格的复杂报表、跨页丢失错误等
3. 版面泛化弱：多栏、异形排版阅读顺序易错乱，正文标题误判
4. 无法定制图片解析方式，如流程图应转为mermaid，UI原型图应转为视觉交互描述等

解决方案

- 多种模型和策略组合切割解析
- 多页并行提速
- 兼顾性能、成本、准确率

效果：73.6% → 95%

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	文件	原文件截图	分类	优先级	问题描述	问题类	解析	是否通过	解析	是否通过	paddlerOCR解析	padOCR是否通过	gemini是否通过
2	表格+删除线		表格解析	P0	带有合并单元格的复杂表格		正常渲染有合并单元格的表格	是	只提取表格内容，不渲染表格 这个解析方式就是不渲染表格	是	正常渲染有合并单元格的表格	是	否
3	[redacted]		图片解析	P0	简单流程图		有图片有解说，文案解说有逻辑性较好，但是会有一些多余的线条，相比没有paddlerOCR好	是	图片解说只做文案提取，没有逻辑解读	是	有图片有解说，文案解说有逻辑性较好	是	是
4	表格+删除线		文案识别	P2	文案识别错误，例如P0识别成PO	textin本身问题	文案识别错误，0 识别成 O http://projectmanage.netease-official.lcap.163yun.com/dashboard/BugDetail?id=3341762000282880	否	P0正常识别	是	P0正常识别	是	大多数符合预期
5			图片解析	P0	1、解析效果：流程图解析错误，流程关系解析后丢失	textin本身问题	流程图图片不显示，并且解析内容是一个表格 有问题，已提单 缺陷单：流程图解析错误，把流程关系都解析没了 http://projectmanage.netease-official.lcap.163yun.com/dashboard/BugDetail?id=3332564519390208	否	https://dify.codewave-test.163yun.com/agentportal/console/datasets/Detail/chunk?knowledgeIdentifier=ad38bba4c8e444b99a79f3	否	存在和textin一样的问题，流程图图片不显示，并且解析内容是一个表格 https://dify.codewave-test.163yun.com/agentportal/console/datasets/Detail/chunk?knowledgeIdentifier=f88b264a67da41c28c5805d4a241e65e&documentId=26779018	否	是
6	采购项目第三页		图片解析	P0	2. 复杂流程图解析后图片丢失且有乱码	textin本身问题	有问题，已提单 【文档解析】客户案例-采购项目，原始文档解析为解析文档后，对应的图片信息变成了表格说明，并且有文字乱码，预期是先有图片，再展示文字说明 http://projectmanage.netease-official.lcap.163yun.com/dashboard/BugDetail?id=3334545148587008	否	图片和解说正常	是	对应的图片信息变成了表格说明 https://dify.codewave-test.163yun.com/agentportal/console/datasets/Detail/chunk?knowledgeIdentifier=f88b264a67da41c28c5805d4a241e65e&documentId=26779017	否	是
7	表格删除线.pdf		文案识别	P2	3. 带删除线的文字是否能识别？目前依然有识别到	textin本身问题	带有下划线，删除线的文案依然正常识别文字，分块中不显示下划线和删除线 原始文档划了删除线，标注暂时不实现，但解析文档中又把这部分内容加回来了 http://projectmanage.netease-	否	带有下划线，删除线的文案依然正常识别文字，分块中不显示下划	否	pdf中带有下划线和删除线的文案不显示，但是会只显示下划线	否	

结论 解析效果对比-最新 90%

02

如何让大规模 Agent 长程任务高质量运行？

增强反馈、分治思想和封装变化

Code Agent 无法高质量运行的主要现象和解决思路

未经充分验证、质量不过关

Agent 写完文档或代码，有质量问题。没有测试或用伪造测试来欺骗用户，而功能实际不可用。

上下文窗口受限、中间环节遗漏或篡改需求

- 大规模任务多轮执行，遗忘上下文
- 技术设计、任务拆解等环节遗漏或窜改需求，导致后面代码生成严重偏离路线。

擅自跳过步骤、随意检查通过

- 对于给定的计划列表，AI 不遵循，随意跳步骤
- 粗略检查或每次临时写一段脚本检查，概率性犯错

Code Agent 无法高质量运行的主要现象和解决思路

未经充分验证、质量不过关

Agent 写完文档或代码，有质量问题。没有测试或用伪造测试来欺骗用户，而功能实际不可用。

增强反馈控制、增加闭环

上下文窗口受限、中间环节遗漏或篡改需求

- 大规模任务多轮执行，遗忘上下文
- 技术设计、任务拆解等环节遗漏或窜改需求，导致后面代码生成严重偏离路线。

分治思想

擅自跳过步骤、随意检查通过

- 对于给定的计划列表，AI 不遵循，随意跳步骤
- 粗略检查或每次临时写一段脚本检查，概率性犯错

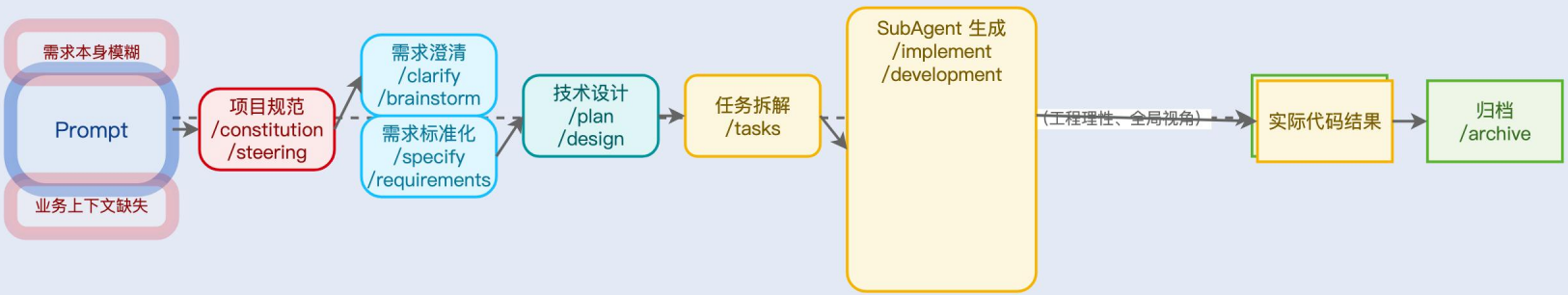
封装变化原则

划分确定性与非确定性的边界

如何应对质量不过关？——增强反馈控制、增加闭环

Harness 的核心 = Guides(Feedforward) + Sensors(Feedback) ——《Harness engineering for coding agent users》

控制，就是在事物多种发展可能性中，依靠反馈不断**缩小可能性空间**，迫使系统向人设定的**目标态收敛**。——金观涛、华国凡《控制论与科学方法论》



通过多维度控制来保证快马不跑偏

前馈控制（Feedforward/Guides）（事前）
（约束规范、执行计划、上下文管理等）

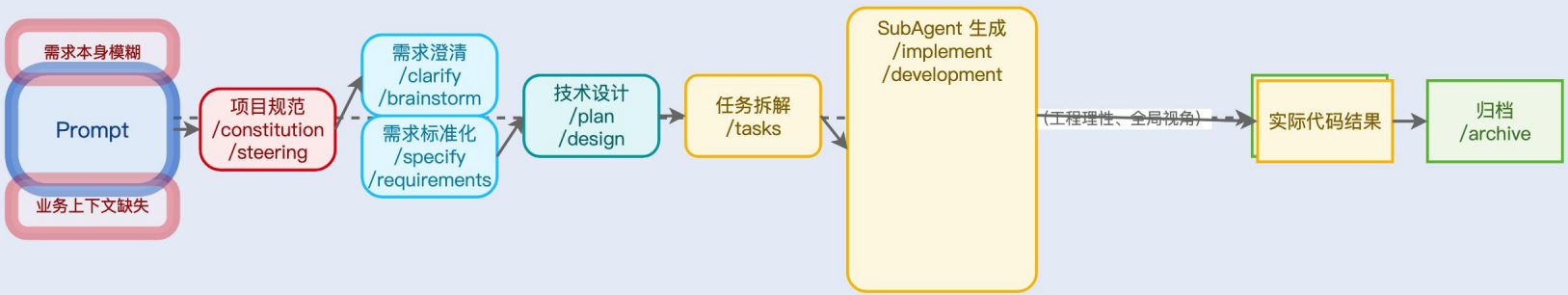
反馈控制（Feedback/Sensors）（事中和事后）
（Linter、Checker、TDD、Hooks 检查等）
（还有 Agent Review）

简单总结
事前可约束、事中可控制、事后可验证

如何应对质量不过关？——增强反馈控制、增加闭环

Harness 的核心 = Guides(Feedforward) + Sensors(Feedback) ——《Harness engineering for coding agent users》

控制，就是在事物多种发展可能性中，依靠反馈不断**缩小可能性空间**，迫使系统向人设定的**目标态收敛**。——金观涛、华国凡《控制论与科学方法论》



纯 Spec Driven是**前馈控制（开环）**：

对于大模型是“软约束”，非常依赖大模型的指令遵循能力。

反馈控制（闭环）：

1. **AI无疲劳性**，可以秒/分钟级完成“生成–报错–修正”**多次循环**，自动收敛问题。
2. 通过软件测试、Lint**形式化验证的可判定性**，大幅剪枝缩小解空间，快速收敛问题。

通过多维度控制来保证快马不跑偏

前馈控制（Feedforward/Guides）（事前）
（约束规范、执行计划、上下文管理等）

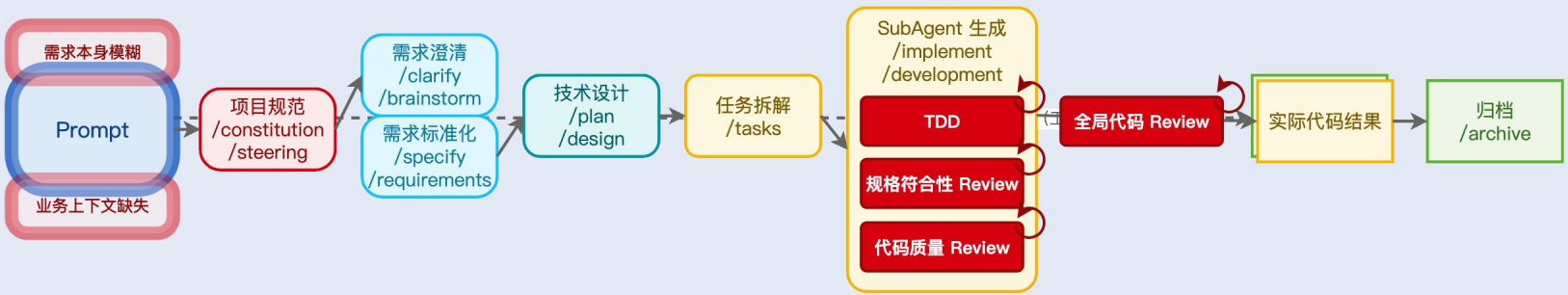
反馈控制（Feedback/Sensors）（事中和事后）
（Linter、Checker、TDD、Hooks 检查等）
（还有 Agent Review）

简单总结
事前可约束、事中可控制、事后可验证

如何应对质量不过关？——增强反馈控制、增加闭环

Harness 的核心 = Guides(Feedforward) + Sensors(Feedback) ——《Harness engineering for coding agent users》

控制，就是在事物多种发展可能性中，依靠反馈不断**缩小可能性空间**，迫使系统向人设定的**目标态收敛**。——金观涛、华国凡《控制论与科学方法论》



纯 Spec Driven是**前馈控制（开环）**：

对于大模型是“软约束”，非常依赖大模型的指令遵循能力。

反馈控制（闭环）：

1. **AI无疲劳性**，可以秒/分钟级完成“生成–报错–修正”**多次循环**，自动收敛问题。
2. 通过软件测试、Lint**形式化验证的可判定性**，大幅剪枝缩小解空间，快速收敛问题。

通过多维度控制来保证快马不跑偏

前馈控制（Feedforward/Guides）（事前）
（约束规范、执行计划、上下文管理等）

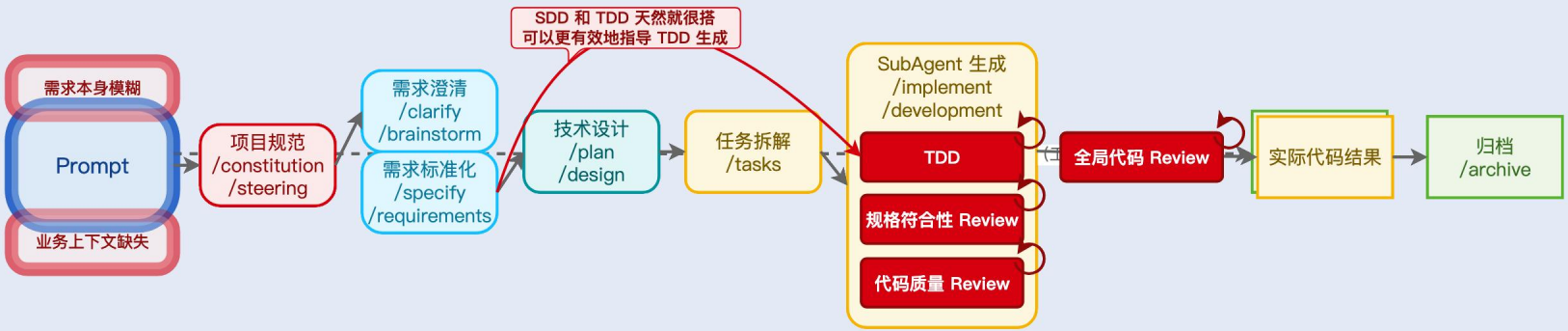
反馈控制（Feedback/Sensors）（事中和事后）
（Linter、Checker、TDD、Hooks 检查等）
（还有 Agent Review）

简单总结
事前可约束、事中可控制、事后可验证

如何应对质量不过关？——增强反馈控制、增加闭环

Harness 的核心 = Guides(Feedforward) + Sensors(Feedback) ——《Harness engineering for coding agent users》

控制，就是在事物多种发展可能性中，依靠反馈不断**缩小可能性空间**，迫使系统向人设定的**目标态收敛**。——金观涛、华国凡《控制论与科学方法论》



纯 Spec Driven是**前馈控制（开环）**：

对于大模型是“软约束”，非常依赖大模型的指令遵循能力。

反馈控制（闭环）：

1. **AI无疲劳性**，可以秒/分钟级完成“生成–报错–修正”**多次循环**，自动收敛问题。
2. 通过软件测试、Lint**形式化验证的可判定性**，大幅剪枝缩小解空间，快速收敛问题。

通过多维度控制来保证快马不跑偏

前馈控制（Feedforward/Guides）（事前）
（约束规范、执行计划、上下文管理等）

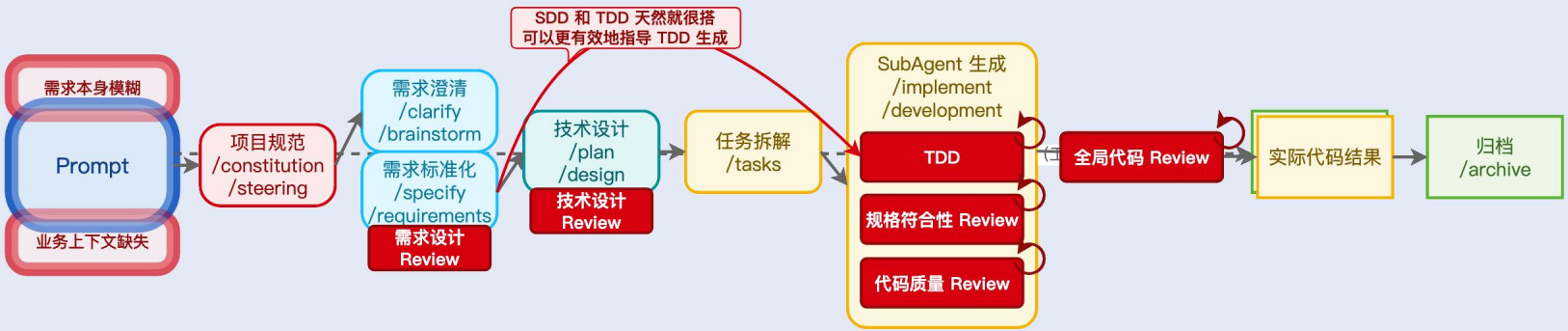
反馈控制（Feedback/Sensors）（事中和事后）
（Linter、Checker、TDD、Hooks 检查等）
（还有 Agent Review）

简单总结
事前可约束、事中可控制、事后可验证

如何应对质量不过关？——增强反馈控制、增加闭环

Harness 的核心 = Guides(Feedforward) + Sensors(Feedback) ——《Harness engineering for coding agent users》

控制，就是在事物多种发展可能性中，依靠反馈不断**缩小可能性空间**，迫使系统向人设定的**目标态收敛**。——金观涛、华国凡《控制论与科学方法论》



纯 Spec Driven是**前馈控制（开环）**：

对于大模型是“软约束”，非常依赖大模型的指令遵循能力。

反馈控制（闭环）：

1. **AI无疲劳性**，可以秒/分钟级完成“生成–报错–修正”**多次循环**，自动收敛问题。
2. 通过软件测试、Lint**形式化验证的可判定性**，大幅剪枝缩小解空间，快速收敛问题。

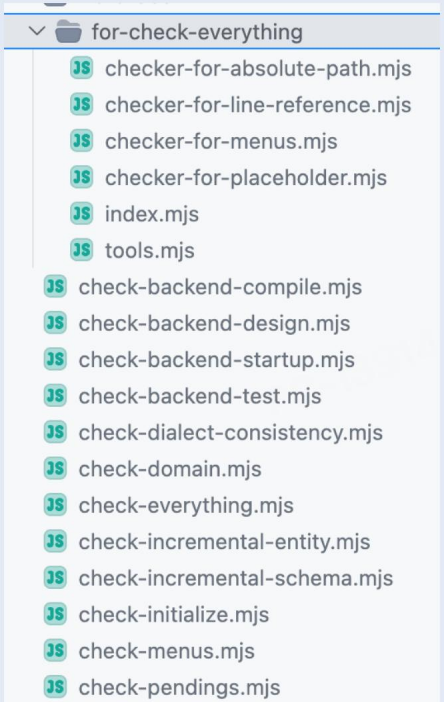
通过多维度控制来保证快马不跑偏

前馈控制（Feedforward/Guides）（事前）
（约束规范、执行计划、上下文管理等）

反馈控制（Feedback/Sensors）（事中和事后）
（Linter、Checker、TDD、Hooks 检查等）
（还有 Agent Review）

简单总结
事前可约束、事中可控制、事后可验证

如何应对质量不过关？——增强反馈控制、增加闭环



上下文窗口问题

模型的上下文会随着编程任务的复杂逐步填满

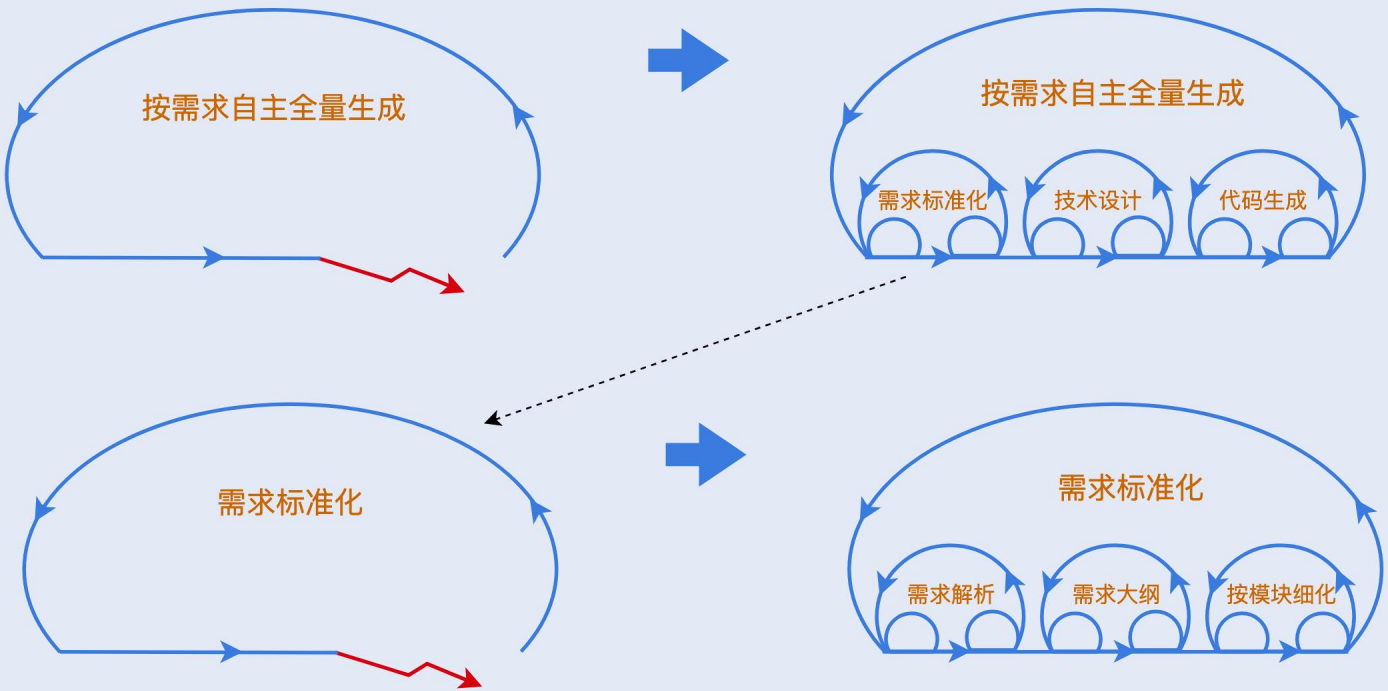
旗舰模型窗口虽已达 1M，但实际 agent 场景普遍在 200K 上下，且有效注意力随填充下降（"lost in the middle"）。一份 PRD 光文本就 5–8 万 token，叠加代码与多轮对话很快超出可用窗口、开始遗忘。

上下文填满后，AI Coding 的效果急剧下降

- **前后矛盾**：对话越长，代码的一致性越差，体现在前端组件的一致性、前后端接口字段的一致性等。
- **功能遗漏**：需求庞大以后，只能实现容易理解的主流程，大量边角需求被遗忘，同时难以检查出来。
- **似是而非**：受限于模型上下文，模型只能以 happy path 首先保证实现效果，全局并不是最优解

分治：将大问题拆解成小问题，将大环拆解成小环和边

- **空间问题**：上下文窗口受限
- **时间问题**：大问题 AI 执行或验证时长不可控，停不下来或不符合预期停止



分治：大需求怎么拆更合理？拆的粒度多大更合适？

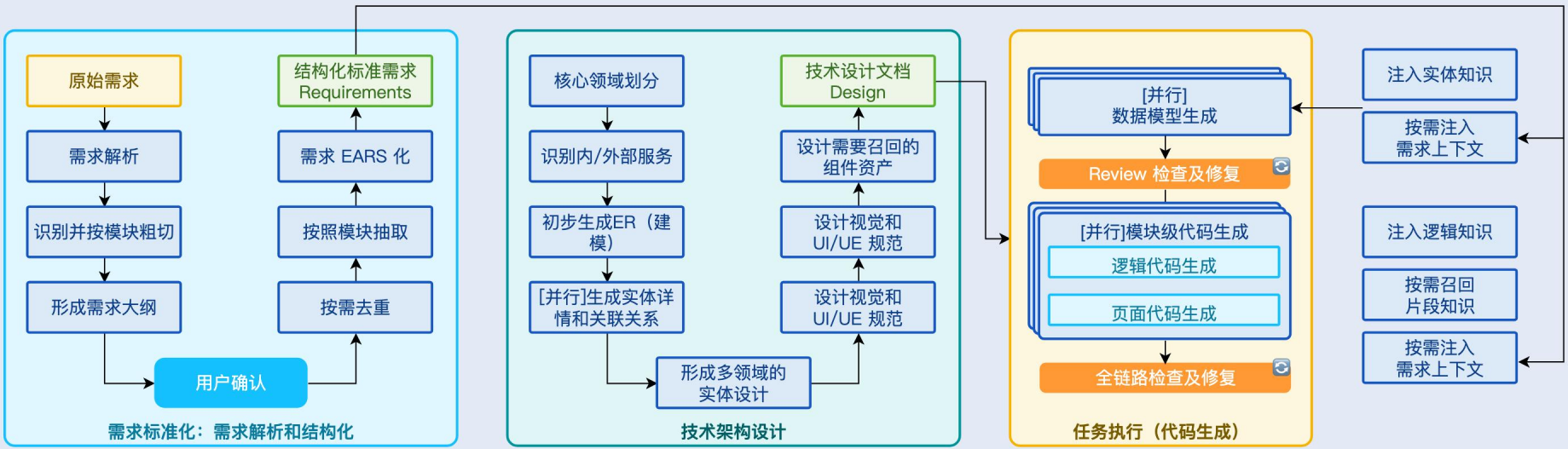
评估维度	功能模块拆解（页面能力目录）	领域边界拆解（DDD）	用户故事拆解（交付增量）
功能清晰	强。功能页面、菜单、操作入口直观	中。业务边界清楚，但用户不一定能直接映射到实际功能页面	中。能表达用户目标，但功能边界容易随故事粒度变化
数据设计	中。能保留页面字段、操作对象和上下游关系，但需要后续统一校准	强。适合梳理业务对象、关系和边界一致性	弱。通常只描述目标，不会完整保留数据结构
功能验收	强。功能入口、操作步骤、结果复查都容易转成验收项	中。可验收业务规则，但不天然包含功能操作路径	中。能验收用户目标，但需要进一步拆成具体页面步骤
用户理解成本	强。用户看到页面能力目录即可判断大方向	弱。需要理解抽象业务边界	强。用户能快速理解目标
主要风险	**过度贴近功能页面，需要做横向一致性校准**	**过早抽象，遗漏页面细节和操作细节**	**易变成愿望清单，缺少实现验收细节**

需求按功能模块拆解切入，并不意味后续不再需要领域边界拆解。相反，它会促进后续链路整合落地：

- 先按功能拆解+标准化会把原始需求沉淀为**更稳定、更干净的中间产物**：功能目录、操作路径和验收清单
- 很多关键信息最初不以抽象概念出现，**标准化后才会呈现**：页面字段、按钮动作、列表结果、详情复查和异常提示
- 避免先按领域边界过早抽象带来的遗漏。让后续链路不是凭空抽象，而是基于**可操作、可验证的业务事实**进行整合。

分治：让软件研发回归合理的架构拆分

“拆”是架构师永远的宿命轮转，无论什么时代。



按功能模块拆解

按领域划分拆解
并理清功能模块与领域划分 N:N 的依赖关系
高内聚、低耦合

大规模编程任务的模块级执行
前端按模块实现、服务端按依赖查找具体领域再实现
渐进式披露和上下文隔离

如何应对擅自跳过步骤？——封装变化划分确定性与非确定性的边界

确定性场景：边运行结束，即达到目标

- 用拧开水龙头的旋转角度控制水的流量
- 把数据传入常用函数得出结果：`y = f(x)`
- 用编程脚本处理信息：`./parse.js input.md`

如何应对擅自跳过步骤？——封装变化划分确定性与非确定性的边界

确定性场景：边运行结束，即达到目标

- 用拧开水龙头的旋转角度控制水的流量
- 把数据传入常用函数得出结果：`y = f(x)`
- 用编程脚本处理信息：`./parse.js input.md`

更适合用开环来控制

如何应对擅自跳过步骤？——封装变化划分确定性与非确定性的边界

确定性场景：边运行结束，即达到目标

- 用拧开水龙头的旋转角度控制水的流量
- 把数据传入常用函数得出结果：`y = f(x)`
- 用编程脚本处理信息：`./parse.js input.md`

更适合用开环来控制

非确定性场景：产物概率输出，很不一定符合预期

- 包含 AI 的功能。典型的概率性输出
- 有人的地方。比如人填写的输入框，再比如团队管理
- 外界环境。比如依赖故障、市场变化、物理因素等
- 异常情况。比如系统内外的bug等。

如何应对擅自跳过步骤？——封装变化划分确定性与非确定性的边界

确定性场景：边运行结束，即达到目标

- 用拧开水龙头的旋转角度控制水的流量
- 把数据传入常用函数得出结果：`y = f(x)`
- 用编程脚本处理信息：`./parse.js input.md`

更适合用开环来控制

非确定性场景：产物概率输出，很不一定符合预期

- 包含 AI 的功能。典型的概率性输出
- 有人的地方。比如人填写的输入框，再比如团队管理
- 外界环境。比如依赖故障、市场变化、物理因素等
- 异常情况。比如系统内外的bug等。

需要通过闭环控制来收敛问题

如何应对擅自跳过步骤？——封装变化划分确定性与非确定性的边界

确定性场景：边运行结束，即达到目标

- 用拧开水龙头的旋转角度控制水的流量
- 把数据传入常用函数得出结果：`y = f(x)`
- 用编程脚本处理信息：`./parse.js input.md`

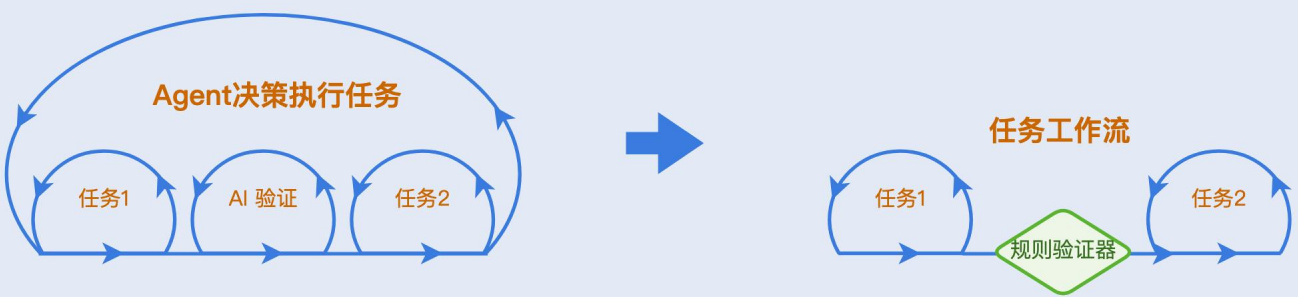
更适合用**开环**来控制

非确定性场景：产物概率输出，很不一定符合预期

- 包含 AI 的功能。典型的概率性输出
- 有人的地方。比如人填写的输入框，再比如团队管理
- 外界环境。比如依赖故障、市场变化、物理因素等
- 异常情况。比如系统内外的bug等。

需要通过**闭环**控制来收敛问题

按封装变化原则划分边界：将确定不变的场景用脚本规则固化处理，变化的让给 AI 处理。



封装变化示例：工作流主导的任务管理器

遇到的问题：需求+技术设计有海量的上下文：40+数据实体，150+服务端逻辑，60+前端页面

- **单Agent纯自主执行**：模型能做完，但往往会丢很多页面细节
- **纯Skill引导**：Skill多了之后，容易进入错误 Skill 或者忽视某些关键 Skill
- **多Agent执行**：Agent之间容易相互捧场、串通一气，最后放大错误

控制策略优化

- 放弃纯自主执行、Skill引导等控制策略，采用工作流+SubAgent的方式：建**任务管理器**，管理各阶段任务

方案优势

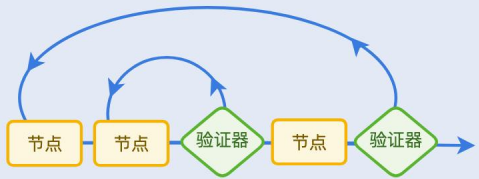
- 任务执行更稳定
- 可以做依赖分析，补充缺失任务。
- 任务独立隔离，方便注入知识，不会造成上下文污染
- 任务可以暂停、重试。起到 Agent 状态恢复、断点继跑的作用。

> 与 Claude Code Dynamic Workflow、Graph Engineering 的思想一致

Loop 工程中几种思想小结汇总

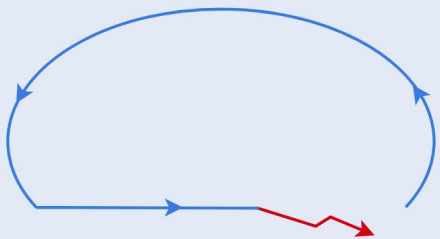
增强反馈控制、增加闭环

你的控制流中不要只有开环/前馈

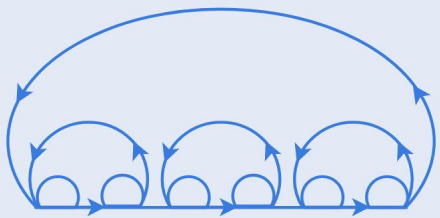


在容易断裂的地方增加闭环/反馈

分治思想



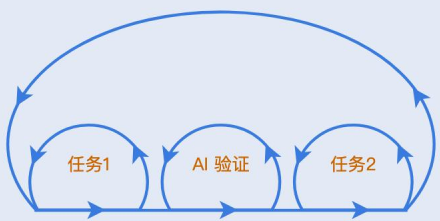
你的控制流不要只有一个大环在循环，很脆弱



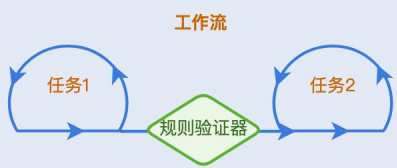
大环拆解成小环和边，由小往大夯实运转

封装变化原则

划分确定性与非确定性的边界



闭环也有成本，并不一定全都要是环



该用环的地方用环，该用边的地方用边

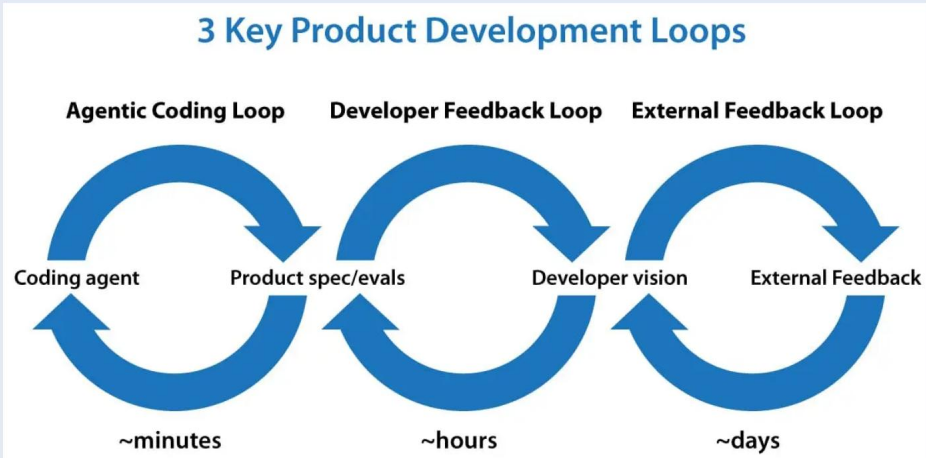
04

如何加速 Code Agent 效果迭代升级？

优化产品研发的迭代流程

从技术 Loop 到产品 Loop

跳出单纯的技术视角，吴恩达在6月底给出了不同的“取景框”，发布文章《Three Key Loops for Building Great Software》，把 Loop 的涉众范围扩大去思考。



基础编程Agent环：

编写代码🔁 测试代码（unit或e2e）

随着编程 Agent 能力的不断提升，目前这个环大多直接被 Claude Code、Open Code 等基础 Agent 支持。

产品研发测试环：

Agent编码完成🔁 产研验收

当编程 Agent 完成一段工作之后，由产研团队人员来验收测试，判断哪些功能和体验需要改进，根据愿景和构想做出进一步的产品决策。

外部用户反馈环：

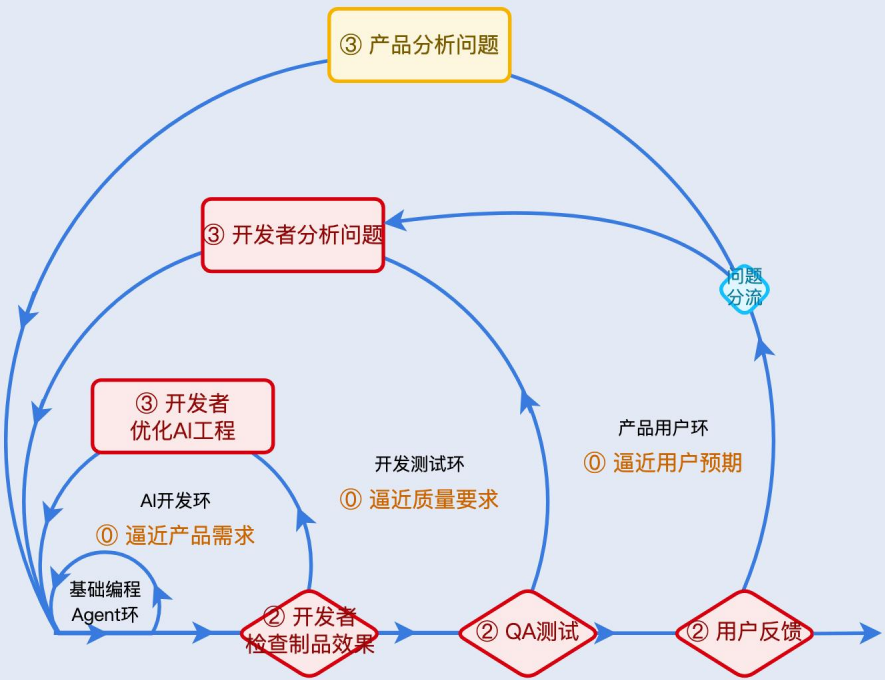
产研上线🔁 用户反馈

产品发布或预发布之后，主动或被动收集用户数据和反馈，回流到产研阶段，进一步优化产品提升用户价值。

基础编程Agent环 → 产品研发测试环 → 外部用户反馈环，**三层嵌套、不同速度、层层驱动**

识别问题（汇总 Loops 图）

在上半年 CodeWave SDD 产品已初步成型，Agent产品使用体验很差，AI概率性高频犯各种错，开发者也不知道往什么方向优化。



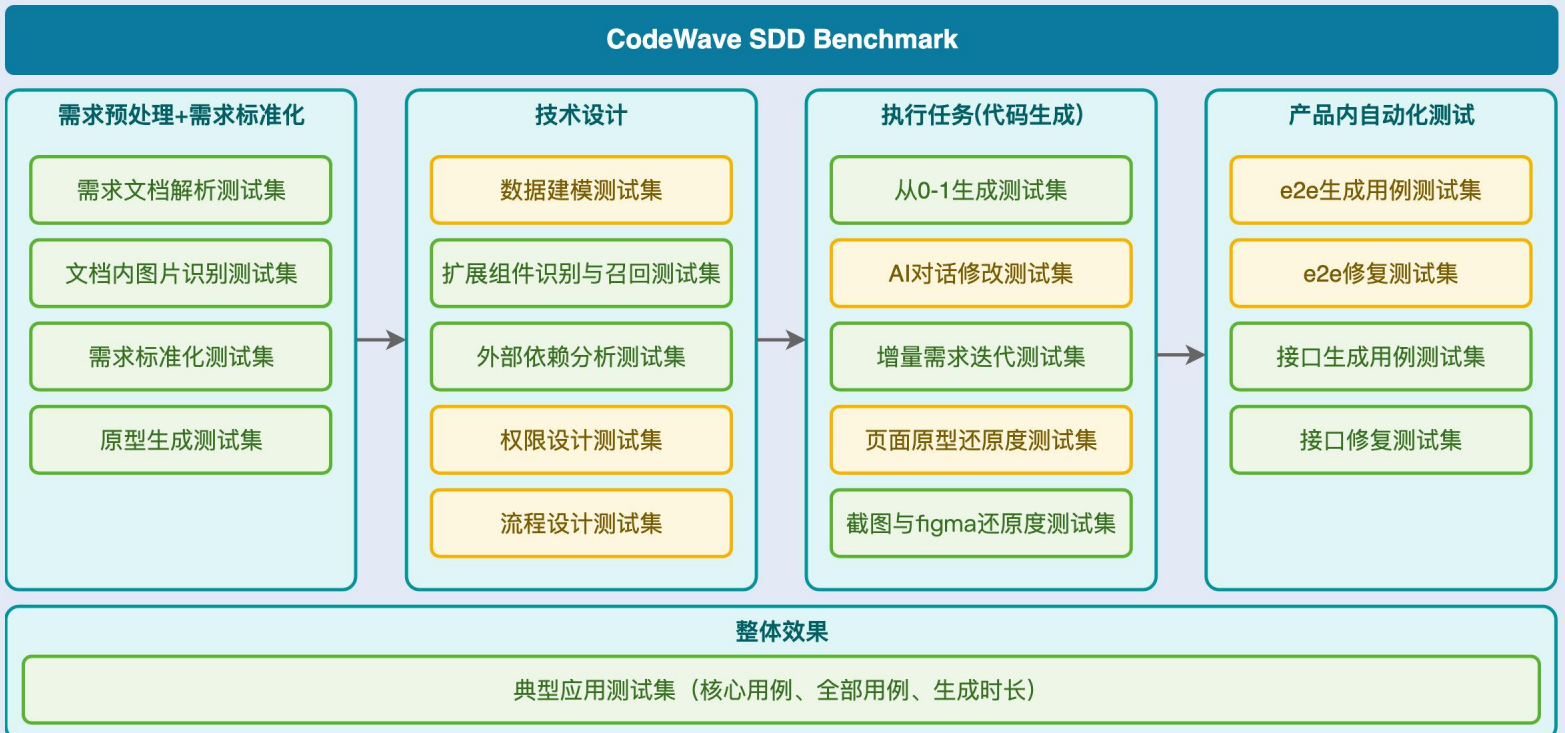
产品迭代视角

利用产品迭代视角来汇总环，识别出了以下**问题（在图中标红）**（从内向外）：

1. AI Agent 技术实现灵活，往往有多种方案，但开发者又不知道该往哪个方向优化。
2. 开发者对自己实现的效果心里也没底。
3. QA测试也没底，能爆出各种bug，但开发者优化后回归时，无法判断是更好还是更差。
4. QA测试以及后来线上用户反馈的问题爆增（Agent产品的痛点，并非传统的代码bug，而是有很多大模型超出预期的情形），开发者分析问题缓慢，无法及时修复。
5. 用户反馈问题多，有时有很多阻塞问题或低级错误。

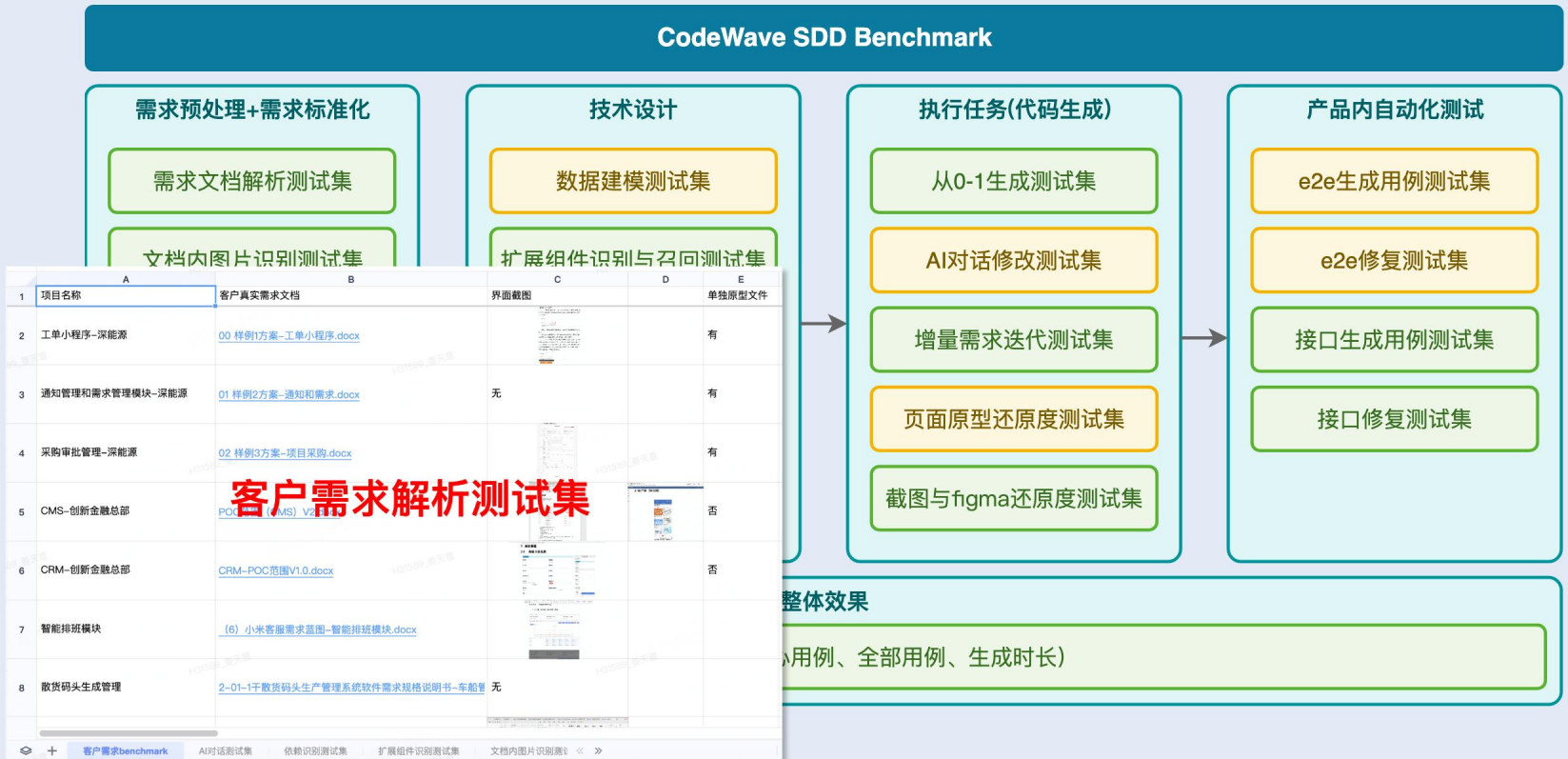
1. 如何应对测试薄弱、心里没底？——No Data No BB

~~结果只能感性判断，缺少数据抓手~~。建立 Benchmark 的关键在**积累代表性用例×有效量化评估**。



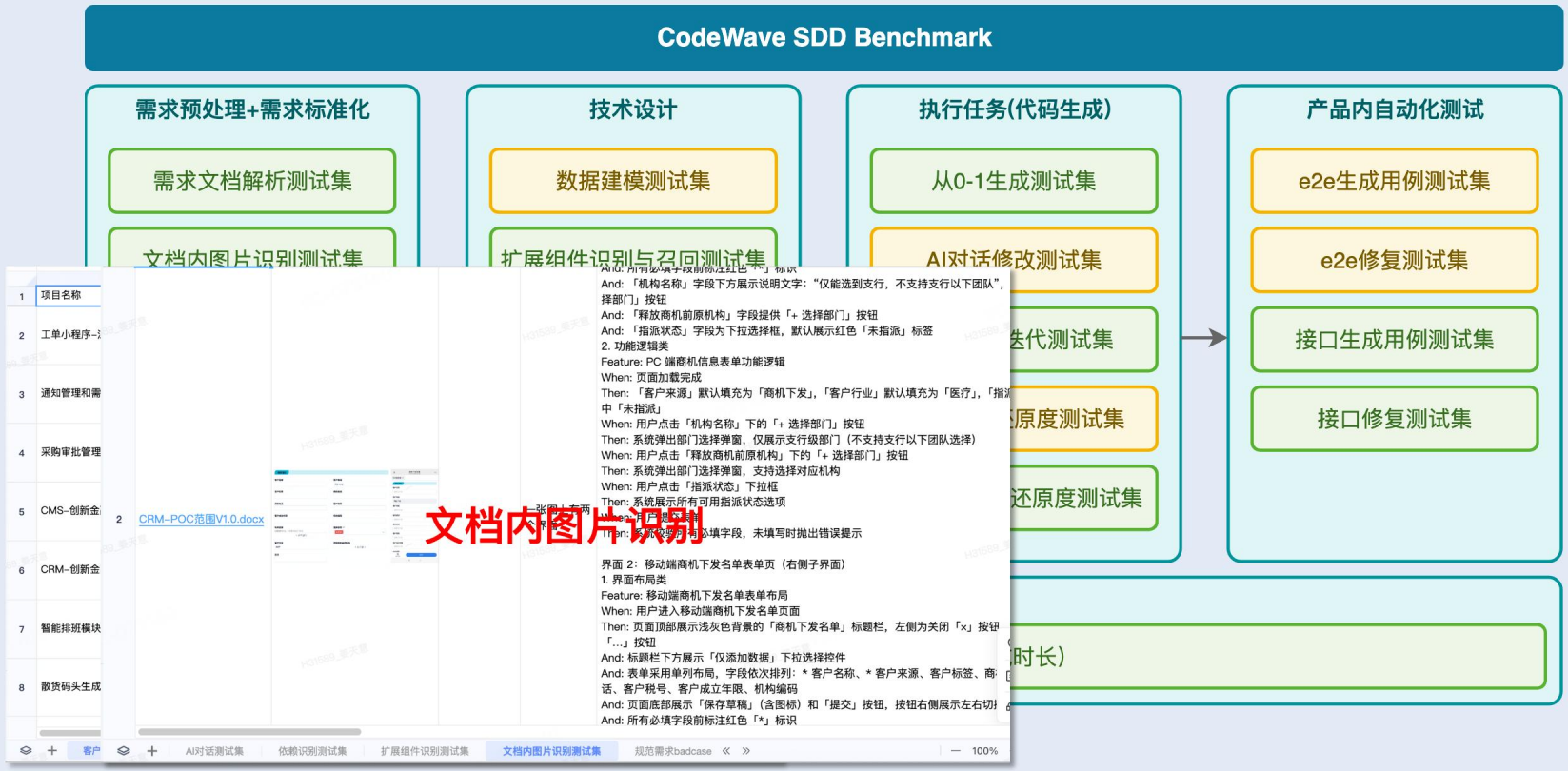
1. 如何应对测试薄弱、心里没底？——No Data No BB

~~结果只能感性判断，缺少数据抓手~~。建立 Benchmark 的关键在**积累代表性用例×有效量化评估**。



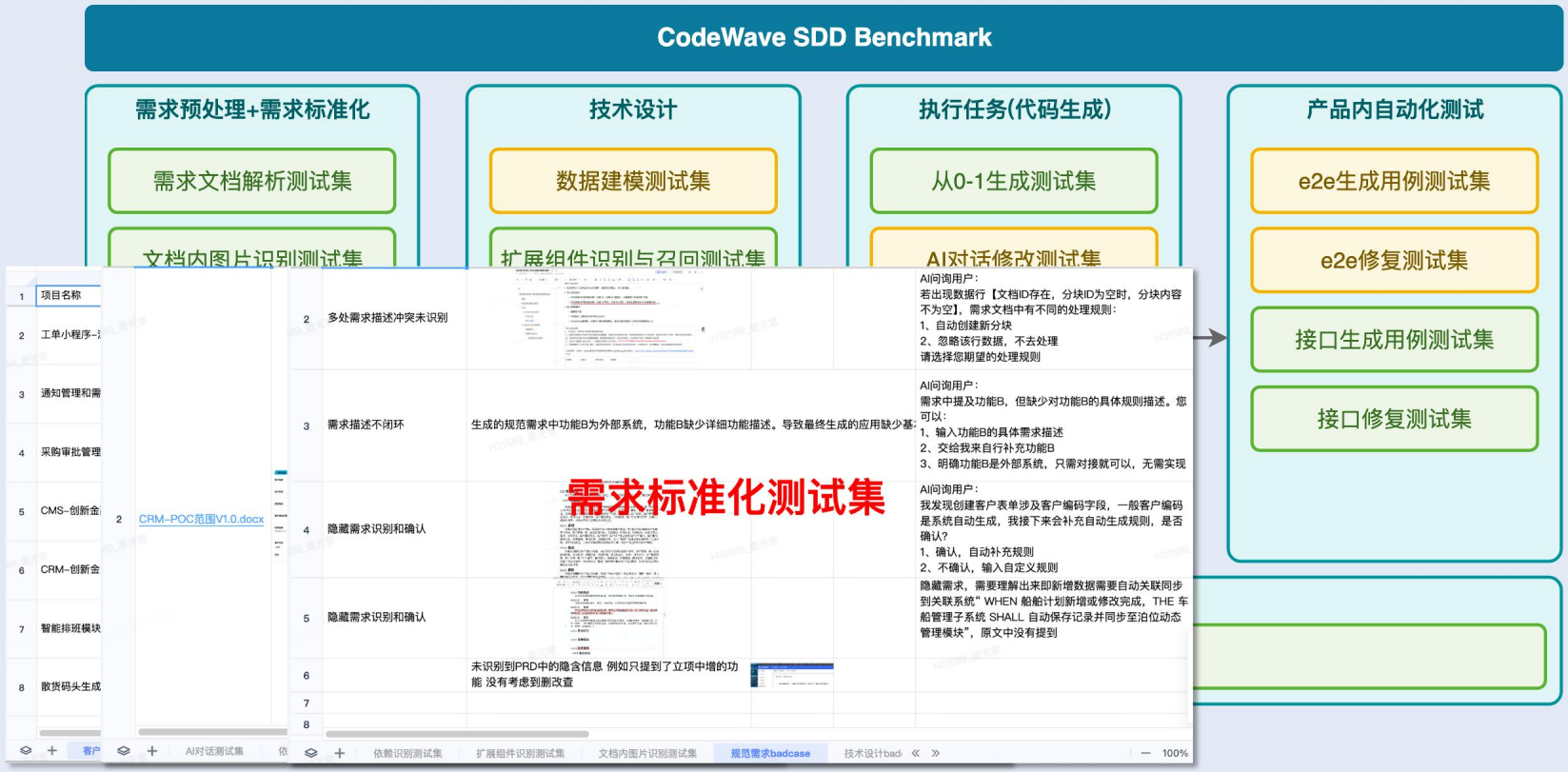
1. 如何应对测试薄弱、心里没底？——No Data No BB

~~结果只能感性判断，缺少数据抓手~~。建立 Benchmark 的关键在积累代表性用例×有效量化评估。



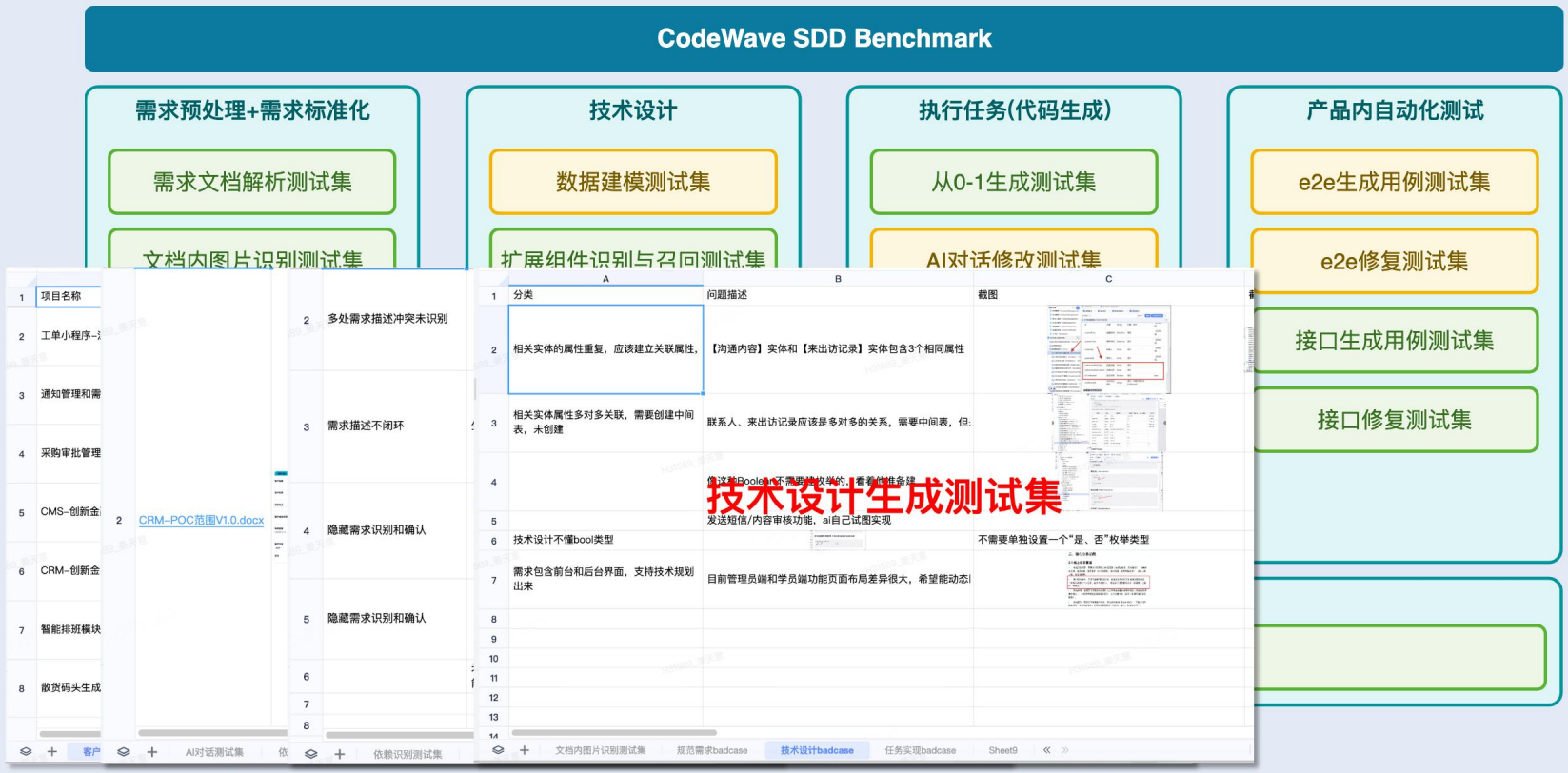
1. 如何应对测试薄弱、心里没底？——No Data No BB

~~结果只能感性判断，缺少数据抓手~~。建立 Benchmark 的关键在**积累代表性用例×有效量化评估**。



1. 如何应对测试薄弱、心里没底？——No Data No BB

~~结果只能感性判断，缺少数据抓手~~。建立 Benchmark 的关键在**积累代表性用例×有效量化评估**。

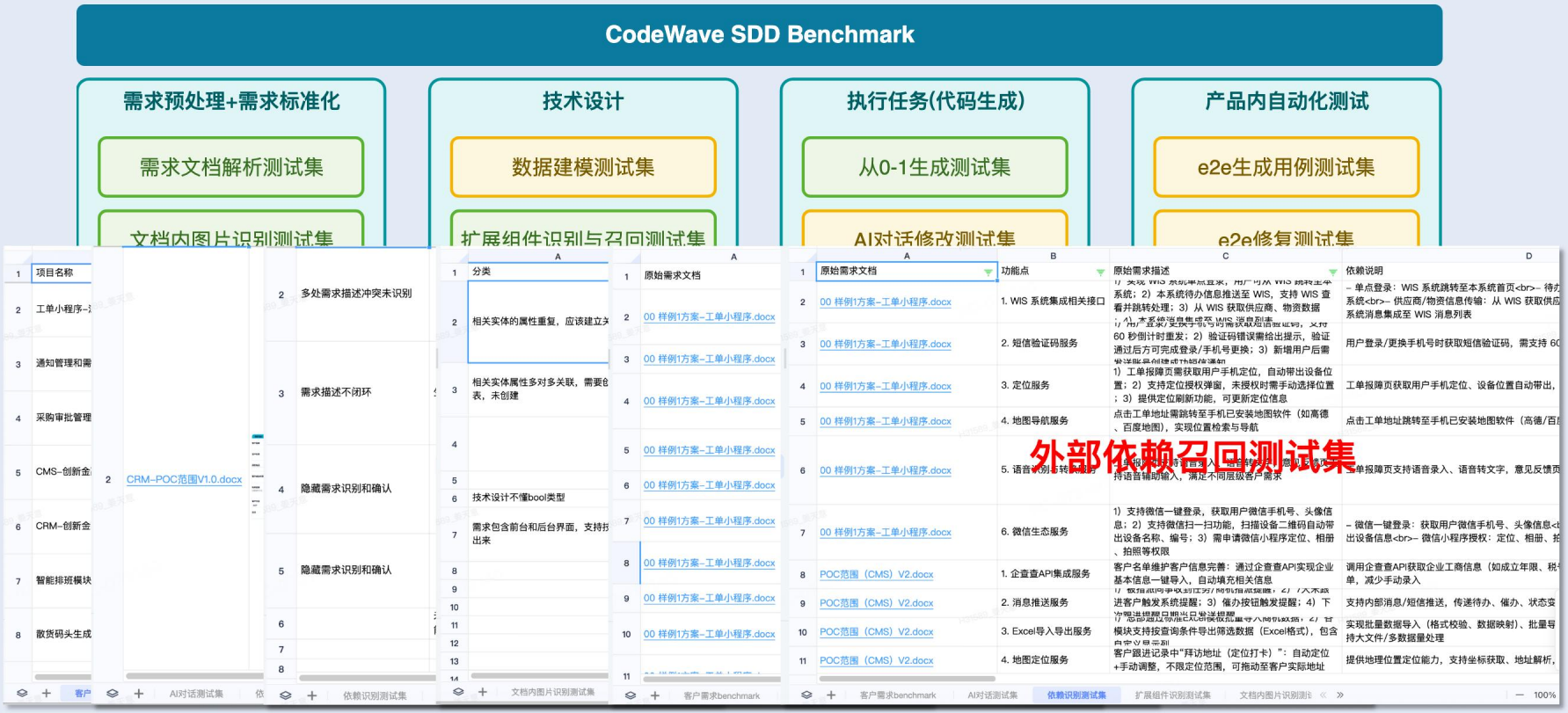


1. 如何应对测试薄弱、心里没底？——No Data No BB

~~结果只能感性判断，缺少数据抓手~~。建立 Benchmark 的关键在积累代表性用例×有效量化评估。

1. 如何应对测试薄弱、心里没底？——No Data No BB

~~结果只能感性判断，缺少数据抓手~~。建立 Benchmark 的关键在**积累代表性用例×有效量化评估**。



1. 如何应对测试薄弱、心里没底？——用例基准(Benchmark)化、结果可量化——有效量化评估机制

Benchmark	评估指标	评估策略
数据建模 Benchmark	准确率	LLM as Judge，通过检查点打分（Rubric-based）
需求标准化设计 Benchmark	准确率	LLM as Judge，通过检查点打分（Rubric-based）
AI对话修改 Benchmark	通过率	SWE Style，每个用例包括上下文环境、输入、评估三要素
页面原型还原 Benchmark	还原度（diff 匹配率）	Hybrid：脚本算法+AI匹配的综合方式。
需求文档解析 Benchmark	多维度评分：文本内容、表格、流程图、文档块顺序等	Rule-based：输入输出确定，不需要AI，类似编辑距离纯规则匹配

2. 如何应对方案灵活、优化方向迷茫？——小成本开发试错、策略可切换可组合

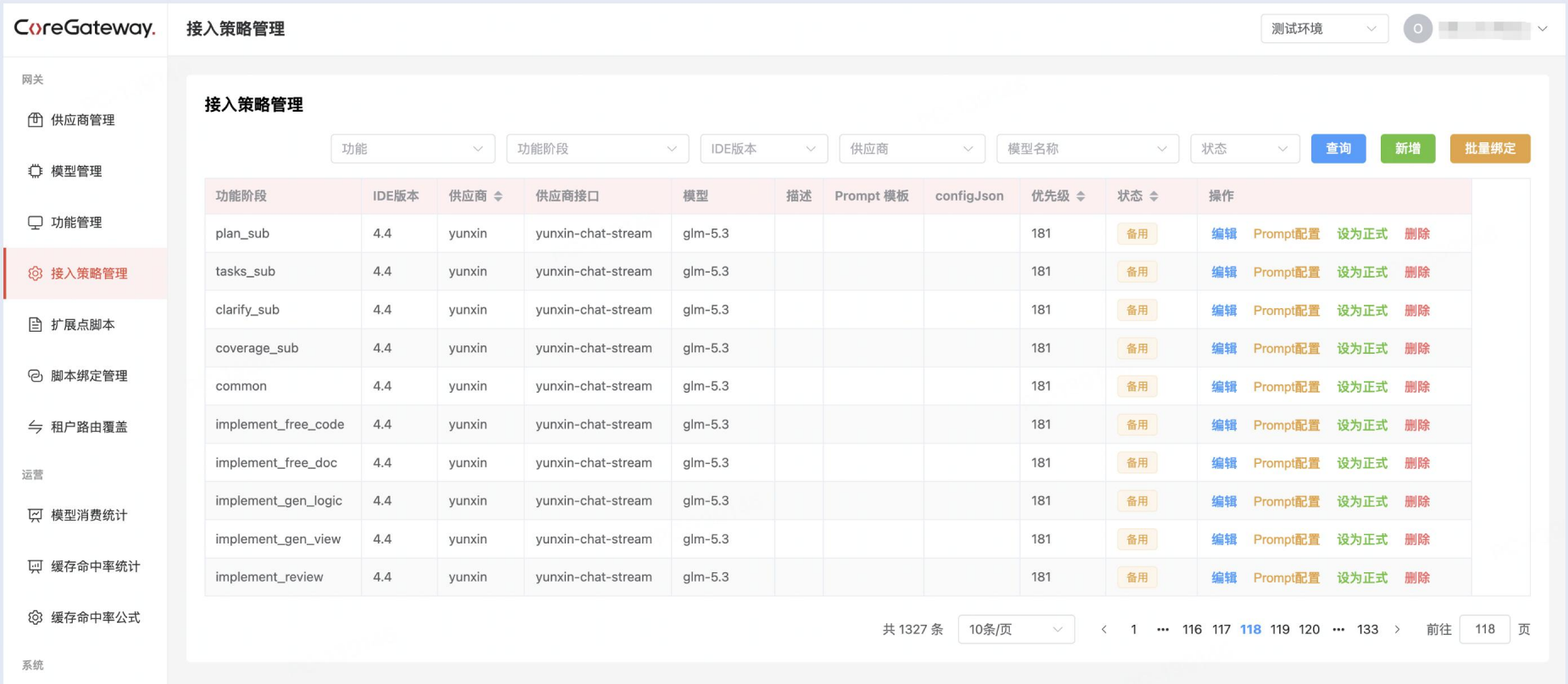
AI Agent 整体是一个不确定性（概率性）很强的系统。在技术评审中对上下文工程等无法做逻辑推演和详细设计。

经过实践，AI 类功能更适合采取快速试错、结果优先的方式。

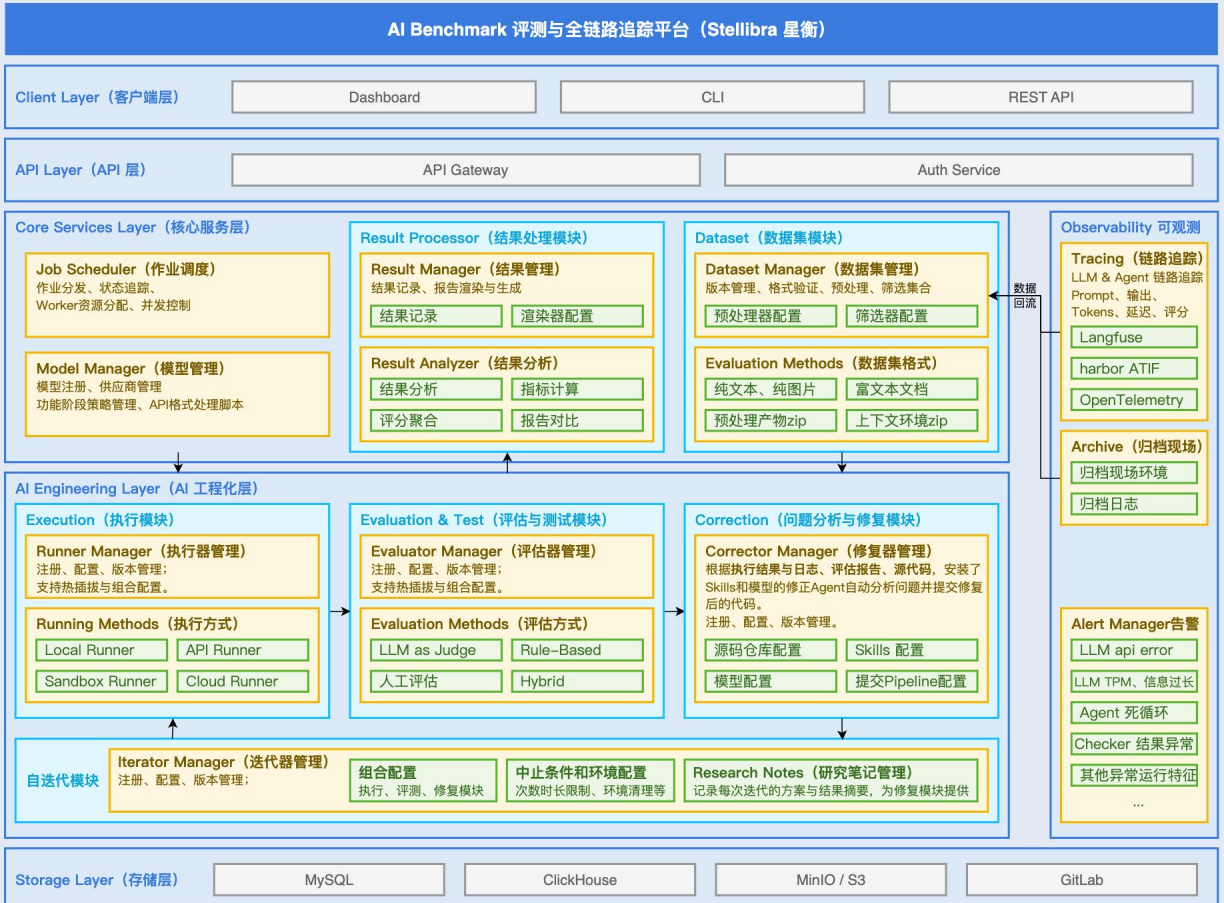
以下是我们在产品 Agent 演进过程中实践的各种试错项和策略项：

试错项和策略项	方案
提示词和沙箱模板	热更新或快速集成
AI网关层面	各阶段模型在线可热切换，thinking开关+强度可调整
Agent决策策略	纯自主探索、Skill驱动、Workflow+SubAgent等
生成权限功能的策略	导入模板、学习模板、自主生成
底层前端原型框架	纯HTML、Vue3+ElementPlus
...	...

2. 如何应对方案灵活、优化方向迷茫？
——小成本开发试错、策略可切换可组合



3. 如何应对产品迭代周期缓慢？——推进自动化智能化设施、降低人工含量



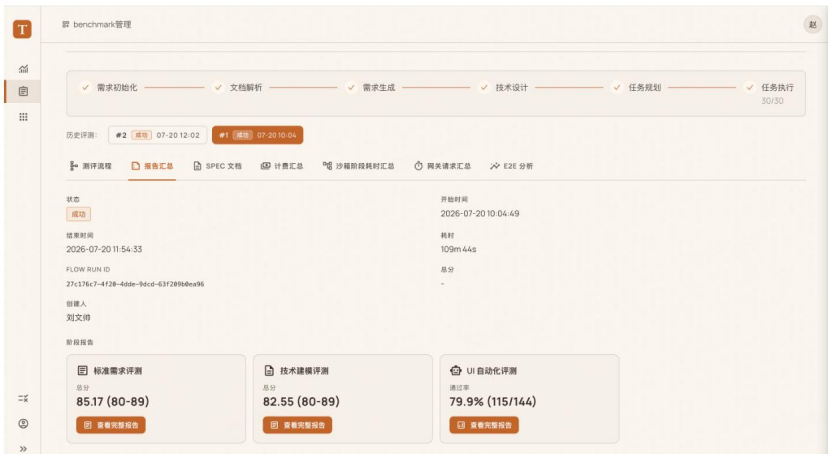
3. 如何应对产品迭代周期缓慢？——推进自动化智能化设施、降低人工含量

(1) 数据集管理（Dataset Manager）

早期，Benchmark 数据集用 Excel 管理

现在，逐渐迁移到系统（纯文本、图片、富文本文档、上下文环境zip）

同时支持数据预处理和筛选器筛选等功能。



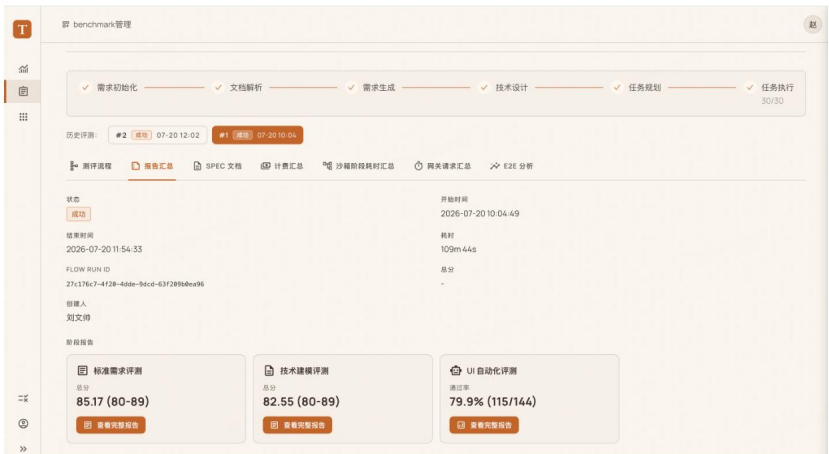
3. 如何应对产品迭代周期缓慢？——推进自动化智能化设施、降低人工含量

(1) 数据集管理（Dataset Manager）

早期，Benchmark 数据集用 Excel 管理

现在，逐渐迁移到系统（纯文本、图片、富文本文档、上下文环境zip）

同时支持数据预处理和筛选器筛选等功能。

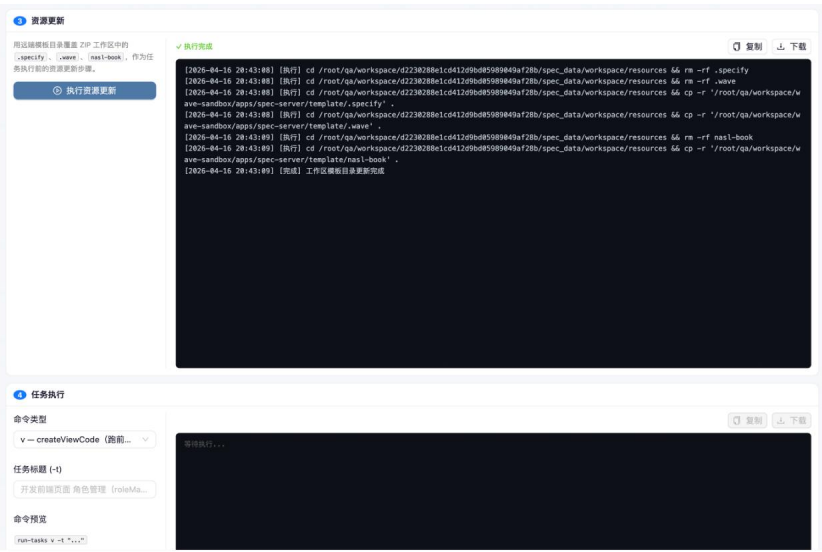


(2) 自动化执行（Auto Execution）

为适配评测增强了以下功能：

- 整体应用和各阶段均可执行
- 已有现场断点继续执行
- 执行入口要以 API 或脚本暴露，通过 Runner 接入

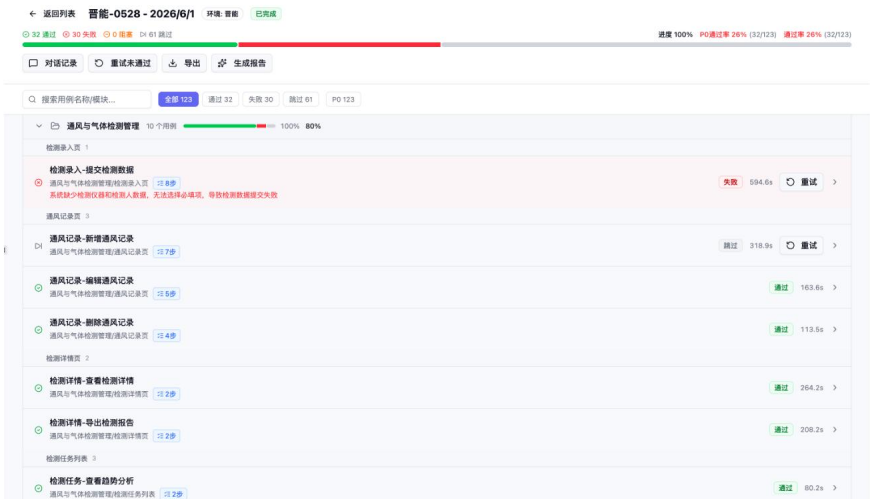
目前团队主要以 Local Runner 和 API Runner 为主。



3. 如何应对产品迭代周期缓慢？——推进自动化智能化设施、降低人工含量

(3) 自动化测试与评估（Auto Test & Evaluation）

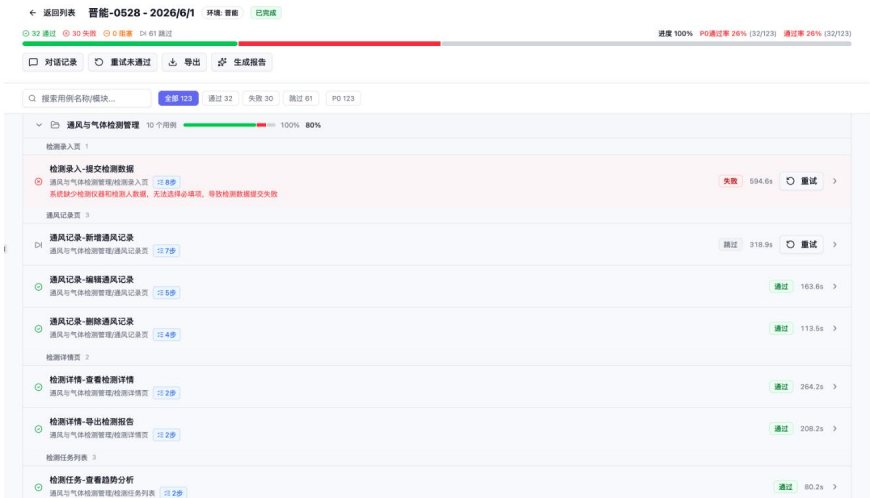
- 整体应用按e2e做自动化测试
- 系统提供了一键测试、生成报告的功能。



3. 如何应对产品迭代周期缓慢？——推进自动化智能化设施、降低人工含量

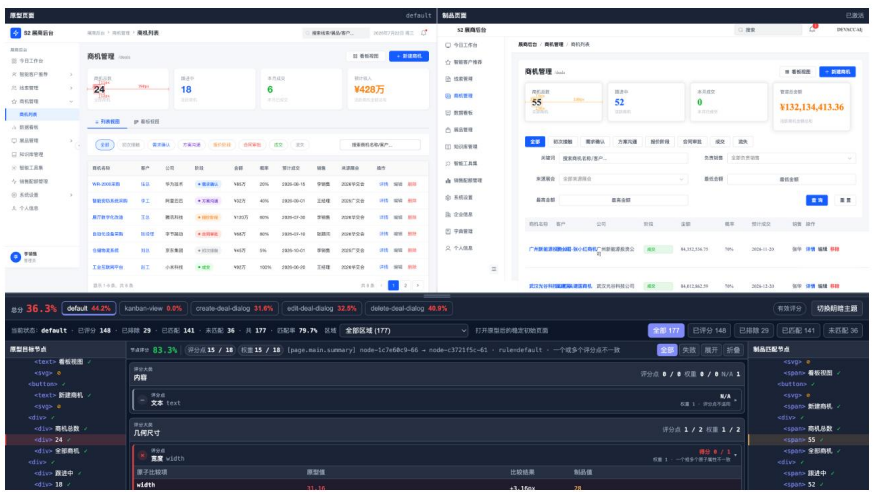
(3) 自动化测试与评估（Auto Test & Evaluation）

- 整体应用按e2e做自动化测试
- 系统提供了一键测试、生成报告的功能。



各阶段 Benchmark 按照各自的评估方式（前面的表格）提供相应的评估器接入系统。

下图为页面原型还原评估器自动评估后产生的报告。



3. 如何应对产品迭代周期缓慢？——推进自动化智能化设施、降低人工含量

(4) 自动化分析问题与修复（Auto Correction）

开发者日常本地使用Agent，沉淀出很多 **Skills**：

- 日志下载 Skill
- 日志分析和 debug 思路 Skill
- 项目开发指南与最佳实践 Skill
- 经验沉淀回流提示词 Skill
- 提交代码/提MR Skill
- 关闭bug单 Skill
- 自动部署测试环境 Skill
- ...

- 很多 bug 能本地 Agent **自闭环修复+人review**
- 将 Skills、源码和 gitlab MR 配置接入系统

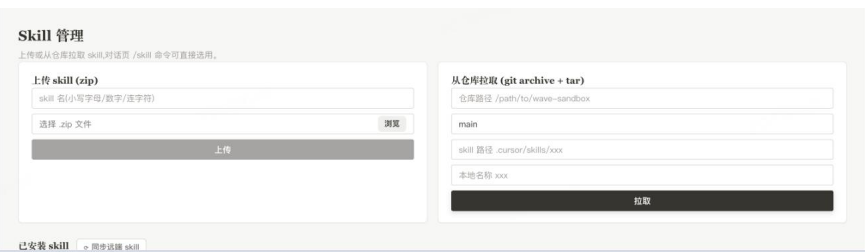
3. 如何应对产品迭代周期缓慢？——推进自动化智能化设施、降低人工含量

(4) 自动化分析问题与修复（Auto Correction）

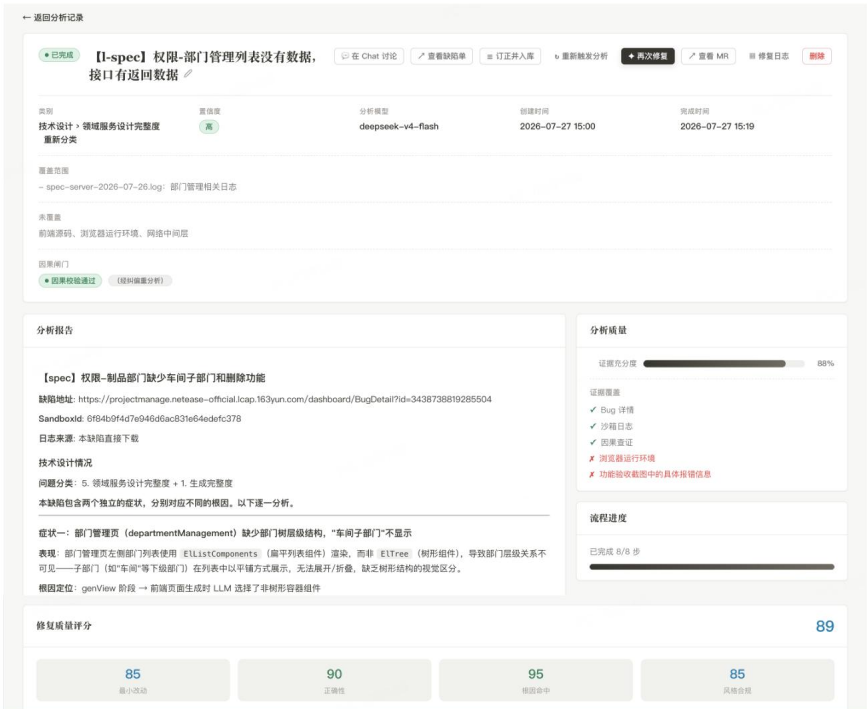
开发者日常本地使用Agent，沉淀出很多 **Skills**：

- 日志下载 Skill
- 日志分析和 debug 思路 Skill
- 项目开发指南与最佳实践 Skill
- 经验沉淀回流提示词 Skill
- 提交代码/提MR Skill
- 关闭bug单 Skill
- 自动部署测试环境 Skill
- ...

- 很多 bug 能本地 Agent **自闭环修复+人review**
- 将 Skills、源码和 gitlab MR 配置接入系统



QA 提 bug 单时顺手点击**一键分析**，等待几分钟后，开发者只需要 review 报告和 MR 即可。



3. 如何应对产品迭代周期缓慢？
——推进自动化智能化设施、降低人工含量

(5) 自我迭代（Auto Iteration）

自我迭代的**前提条件**：如果某阶段的 Benchmark 有**自动化执行、自动化测评与智能化修复**的三大主要设施积累，这条**链路就可以自动化闭环**。

自我迭代**迭代器**，本身实现并不复杂。主要给迭代Agent以下策略进行循环：

- 每次只开发一个**实验想法**，提交一个 commit
- 然后执行+评估后，得到某种**分数**
- **能提升目标分数或能明显简化系统**就保留 commit，否则就 reset 这个提交
- **记录实验笔记**：避免 AI 重复研究开发。也支持人工批注。

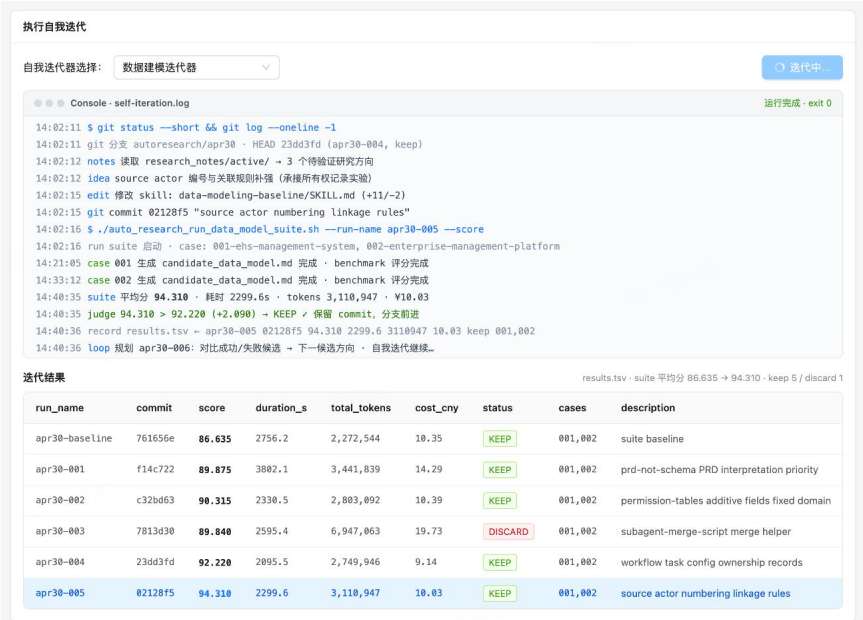
3. 如何应对产品迭代周期缓慢？
——推进自动化智能化设施、降低人工含量

(5) 自我迭代（Auto Iteration）

自我迭代的**前提条件**：如果某阶段的 Benchmark 有**自动化执行、自动化测评与智能化修复**的三大主要设施积累，这条**链路就可以自动化闭环**。

自我迭代**迭代器**，本身实现并不复杂。主要给迭代Agent以下策略进行循环：

- 每次只开发一个**实验想法**，提交**一个 commit**
- 然后执行+评估后，得到某种**分数**
- **能提升目标分数或能明显简化系统**就保留 commit，否则就 reset 这个提交
- **记录实验笔记**：避免 AI 重复研究开发。也支持人工批注。



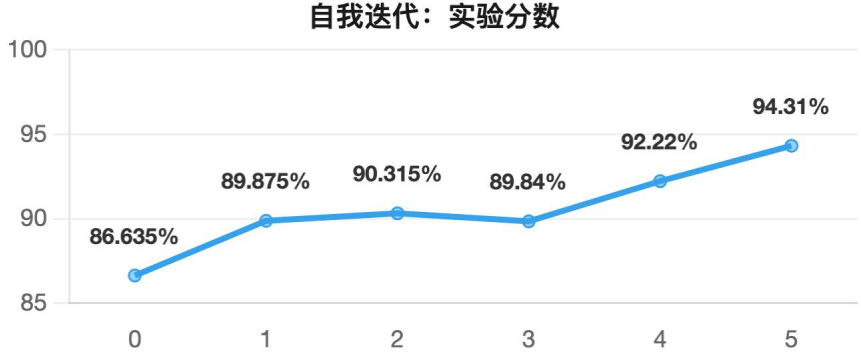
3. 如何应对产品迭代周期缓慢？——推进自动化智能化设施、降低人工含量

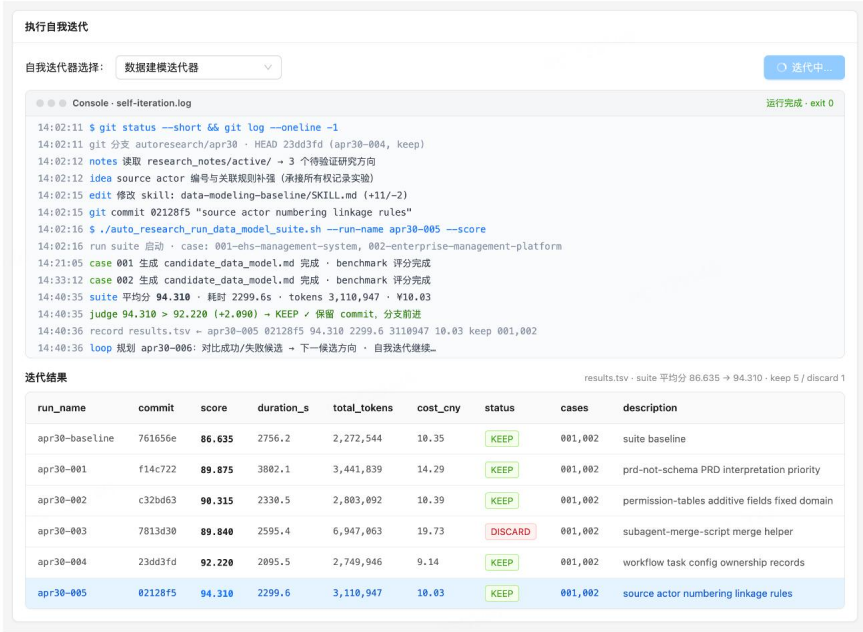
(5) 自我迭代（Auto Iteration）

自我迭代的**前提条件**：如果某阶段的 Benchmark 有**自动化执行、自动化测评与智能化修复**的三大主要设施积累，这条**链路就可以自动化闭环**。

自我迭代**迭代器**，本身实现并不复杂。主要给迭代Agent以下策略进行循环：

- 每次只开发一个**实验想法**，提交**一个 commit**
- 然后执行+评估后，得到某种**分数**
- **能提升目标分数或能明显简化系统**就保留 commit，否则就 reset 这个提交
- **记录实验笔记**：避免 AI 重复研究开发。也支持人工批注。





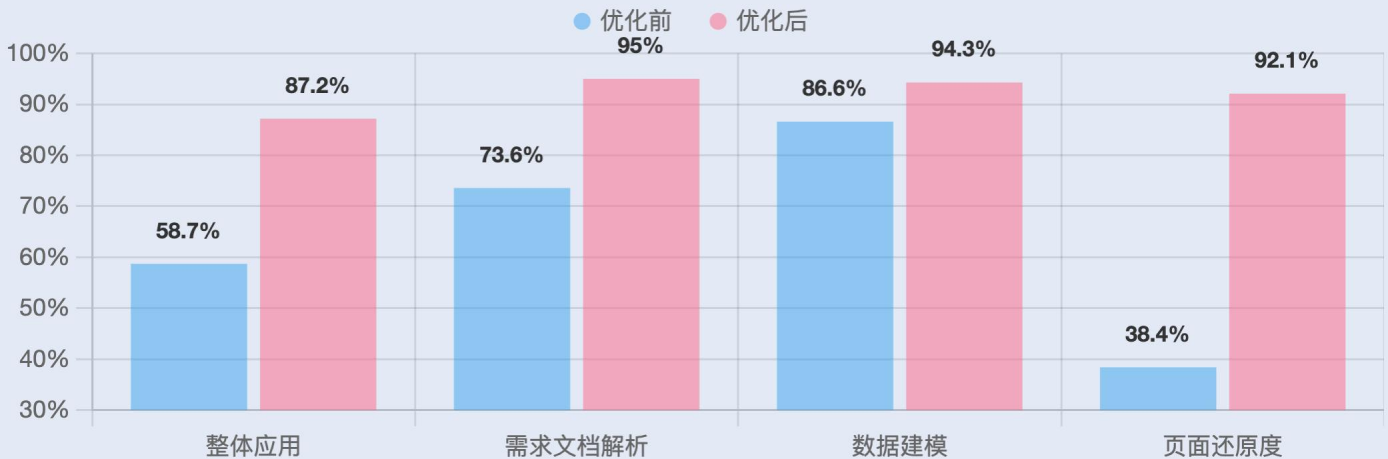
05

总结和展望

产品 Agent 迭代效果

Agent效果提升

Benchmark 指标（对用户体验有明显提升）



Agent迭代效率提升

- 测评启动时长由原来的 **1-3天**，缩短为 **5-15 min** 左右。
- 平均修复时长由原来的 **2-4h** 降低到 **15min** 左右。

生成效果对比

L3 AI工具生成

2.1 功能模块清单

序号	一级模块	二级功能	优先级
1	业务台账概览	业务台账查询与导出	P
2	任务管理	任务总览、待办任务、我的发起、我的已办、任务办理详情	P0
3	客户信息管理	客户信息总览、客户信息详情、客户信息编辑	P0
4	授信管理	授信总览、授信信息详情、授信申请清单、授信申请详情	P0
5	反担保管理	保证信息管理、保证信息详情、抵质押信息管理、抵质押信息详情	P1
6	业务办理	业务总览、单笔业务启用申请、保函核准申请、签署债券发行书申请、债券发行登记、借据登记	P0
7	五级分类管理	风险分类情况总览、风险分类认定申请、风险情况报告	P1
8	保后检查	保后检查（现场）、保后检查（非现场）	P1
9	风险预警	风险预警总览、风险预警申请	P1
10	解保管理	解保信息总览、解保申请	P1
11	保费管理	保费计划管理与保费缴纳	P1
12	[illegible]	部门管理、角色管理、用户管理、权限管	P0
13	登录集成	OA系统单点登录	P0
14	消息中心	消息发送记录查询与重新发送	P1

子菜单全部丢失

会发现只做了一级菜单，子菜单全部丢失，所有功能均缩水严重

生成效果对比

L3 AI工具生成

2.1 功能模块清单

序号	一级模块	二级功能	优先级
1	业务台账概览	业务台账查询与导出	P
2	任务管理	任务总览、待办任务、我的发起、我的已办、任务办理详情	P0

子菜单全部丢失

CodeWave SDD 生成

2. 功能大纲

2.1 功能模块清单

序号	一级模块	二级功能	优先级
1	业务台账概览	业务台账查询与导出	P1
2	任务管理	任务总览、待办任务、我的发起、我的已办、任务办理详情	P0
3	客户信息管理	客户信息总览、客户信息详情、客户信息编辑	P0
4	授信管理	授信总览、授信信息详情、授信申请清单、授信申请详情	P0
5	反担保管理	保证信息管理、保证信息详情、抵质押信息管理、抵质押信息详情	P1
6	业务办理	业务总览、单笔业务启用申请、保函核准申请、签署债券发行书申请、债券发行登记、借据登记	P0
7	五级分类管理	风险分类情况总览、风险分类认定申请、风险情况报告	P1
8	保后检查	保后检查（现场）、保后检查（非现场）	P1
9	风险预警	风险预警总览、风险预警申请	P1
10	解保管理	解保信息总览、解保申请	P1
11	保费管理	保费计划管理与保费缴纳	P1
13	登录集成	OA系统单点登录	P1
14	消息中心	消息发送记录查询与重新发送	P1

首先在功能模块上基本做到100%还原

生成效果对比

划分了两个视图，但完全没有高级搜索区域，也没有操作控制栏

表格操作栏的“详情”等操作项只是弹出消息

生成效果对比

划分了两个视图，但完全没有高级搜索区域

表格操作栏的“详情”等操作项只是弹出消息

客户授信台账视图，高级搜索框完全按照需求规范来做，模糊联想、下拉复选均实现

表格操作栏的“详情”等操作项有详细的抽屉、跳转、表单内容等实现

CodeWave L4 与其他 AI Coding 的对比

特征维度	CodeWave L4	其它 AI Coding 工具
应用多环境部署、一站式管理	✅	claude code CLI、IDE-native 等 L3 工具不具备
云端长程不中断生成，减少人工值守	✅	claude code CLI、IDE-native 等 L3 工具不具备
非技术岗端到端使用	✅	L3 工具不具备，遇到部署等技术问题就阻塞
需求工程，解决复杂企业级需求能力	✅	非 SDD 工具都不具备 speckit、openspec 等工具支撑不了国内企业的复杂场景
需求完整度和效果保证	✅	主要依靠 LLM 发挥，没有确切数据和承诺
国产化、信创要求	✅	国外 L3、L4 工具均不具备

未来展望

增强客户知识的连接与沉淀

工作管线可定制化

端云联动，支持更多技术栈

AI 写代码只是**起点**，

把智能研发落地为企业弹性产能，才是真正的**生产力突破**。

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造

专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践

专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业

专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering

专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理

专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付

专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software